

이 프로젝트와 데이터: RAG 문서를 읽기 전에

1. 개요

이 문서는 개념서 트랙의 첫 문서로, 뒤에 이어지는 문서들이 RAG를 설명할 때 이 프로젝트의 데이터를 예시로 들기 때문에 먼저 필요하다. 공고, 기술 사전, 동시 요구 같은 말이 뒤 문서에서는 설명 없이 등장하므로, 이 문서가 그 바닥을 먼저 깔아 둔다. 이 서비스가 무엇인지, 어떤 데이터를 다루는지, 그 데이터가 어떤 구조인지를 정리하며, RAG를 처음 접하고 이 프로젝트도 처음 보는 독자를 기준으로 적었다.

읽는 순서는 이 문서로 프로젝트와 데이터를 먼저 잡은 다음, `01-rag-basics.md` 부터 RAG 개념으로 넘어가는 식이다.

2. 서비스 개요

채용 공고를 모아 구직자에게 보여주는 서비스로, 여러 채용 사이트에 흩어진 공고를 한곳에 모으고 구직자의 기술 스택을 기준으로 각 공고가 얼마나 맞는지 점수를 매겨 보여준다. 핵심 기능은 이 점수화이며, 지도와 시장 트렌드, AI 챗봇은 그 위에 얹는 서브 기능이다.

뒤 문서들이 다루는 대상인 RAG 챗봇도 이 서브 기능 중 하나이며, 점수화가 주력이고 챗봇은 부가기능이라는 점을 기억해 두면 뒤에서 나오는 "부가기능인데도 진지하게 설계했다"는 서술의 맥락이 잡힌다.

3. 데이터 출처

공고는 여러 공개 채용 공고 소스에서 확보했으며, 국내 소스와 해외 소스로 나뉜다.

출처를 국내와 해외 두 묶음으로 나눈 값을 pool이라 부르며, 국내(domestic)와 해외(global) 소스가 각각 이 값으로 묶인다. 이 pool 값은 원본 공고에 들어 있지 않고 출처를 보고 만들어 넣는 값으로, 구직자가 국내 공고만 보거나 해외 공고만 보고 싶을 때 이 값으로 걸러낸다.

4. 공고 데이터의 구조

수집한 공고 한 건은 정형 데이터로 정리되어 있다. 주요 필드는 다음과 같다.

- 출처와 식별자. 어느 소스의 몇 번 공고인지를 나타내며, 예를 들어 `source:id` 형태로 콜론 앞이 출처, 뒤가 그 소스 안에서의 고유 번호다.
- 제목, 회사, 지역, 마감일 같은 기본 속성.
- 요구 기술 목록. 이 공고가 어떤 기술을 요구하는지를 담는다.
- 요구 자격증 목록.
- 직군 분류. 프론트엔드, 백엔드 같은 구분이다.

원본 텍스트도 따로 보관하는데, 위 필드가 원문에서 뽑아 정리한 값이다 보니 뽑는 과정에서 놓친 정보가 있을 수 있기 때문이다. 그래서 원문 전체를 `raw_posting`이라는 자리에 그대로 남겨 두고, 정리된 값이 이상하면 언제든 원문으로 되돌아가 대조할 수 있게 했다.

5. 엔티티와 관계

이 데이터의 중심은 공고(posting)로, 공고 하나가 여러 대상과 연결되며 그 대상들을 엔티티라 부른다. 엔티티는 각각 자기 테이블을 갖는다.



공고와 엔티티는 다대다 관계인데, 공고 하나가 기술 여러 개를 요구하고 기술 하나가 공고 여러 건에 등장하기 때문에 두 테이블을 직접 잇지 못하고 사이에 링크 테이블을 둔다. `posting_tech` 는 어떤 공고가 어떤 기술을 요구하는지 한 줄에 하나씩 기록한 표이고, 자격증은 `posting_cert` , 직군은 `posting_category` 가 같은 역할을 한다.

6. taxonomy: 표준 기술 사전

같은 기술을 사이트마다 다르게 적는데, 누구는 `React` , 누구는 `React.js` , 누구는 `리액트` 라고 쓴다. 이 셋을 그대로 두면 한 기술이 세 기술로 쪼개져 집계가 어긋난다.

이를 막는 사전이 taxonomy로, 여러 표기를 하나의 표준 이름으로 모아 `React.js` 와 `리액트` 를 모두 `React` 로 잇는다. 표준 이름을 canonical, 그 별칭들을 alias라 부르며, 이 프로젝트의 사전은 표준 기술 447개를 19개 범주로 묶어 관리한다. 자격증도 같은 방식의 사전을 따로 둔다.

기술명을 정리할 때 이 사전을 거치기 때문에, "Python을 요구하는 공고 수"를 세면 `python` , `파이썬` 으로 적힌 것까지 하나로 합쳐 정확한 수가 나온다.

7. co-occurrence: 함께 요구된 기술

한 공고가 여러 기술을 동시에 요구하는데, 어떤 두 기술이 같은 공고에 자주 함께 나오면 그 둘은 현업에서 같이 쓰인다는 신호로 볼 수 있다. `React`와 `TypeScript`가 그런 경우이며, 이렇게 두 기술이 한 공고에 같이 등장하는 것을 동시 요구, 영어로 co-occurrence라 부른다.

이 값은 뒤에서 지식그래프의 뼈대가 되어, "React를 배우면 무엇을 함께 배워야 하는가" 같은 질문에 이 동시 요구 데이터로 답하게 된다. 뒤 문서에서 `Tech CO_OCCURS Tech` 라는 표기가 나오면 바로 이 개념을 가리킨다.

8. mart.db와 Postgres

데이터는 두 곳에 있는데, 이름과 역할을 미리 구분해 두면 뒤 문서를 읽기가 쉬워진다.

- `mart.db`. 수집과 정제가 끝난 정본으로, SQLite 파일 하나이며 중복 제거와 표기 표준화가 여기서 이미 끝났다. 이 프로젝트의 데이터 원천으로 삼는다.
- `Postgres`. 서비스가 실제로 쓰는 데이터베이스로, 백엔드가 여기에 붙고 RAG의 벡터 검색도 여기서 돈다.

mart.db의 데이터를 Postgres로 옮기는 작업이 ETL이며, 구현기 트랙의 01-data-loading.md가 이 작업을 다룬다. 뒤 문서에서 "실데이터"라 하면 이렇게 옮겨진 공고를 뜻한다.

9. 숫자로 보는 규모

이 데이터가 얼마나 큰지 감을 잡는 표이며, 뒤 문서의 실측 수치는 모두 이 규모 위에서 나온다.

항목	값
광고 수	124,237건
국내 광고	약 7,104건
해외 광고	약 117,133건
광고-기술 링크	336,631건
광고-자격증 링크	5,445건
광고-직군 링크	41,849건
표준 기술 수	447개

10. 용어 빠른 사전

뒤 문서에서 설명 없이 나오는 프로젝트 용어를 모았으며, 막히면 여기로 돌아오면 된다.

용어	뜻
광고(posting)	채용 공고 한 건. 이 데이터의 중심 엔티티.
출처(source)	공고를 수집한 소스. 국내 소스와 해외 소스로 나뉜다.
pool	출처를 국내(domestic)와 해외(global)로 나눈 값.
raw_posting	정리 전 원문 텍스트를 그대로 보관한 자리.
엔티티	공고와 연결되는 대상. 기술, 자격증, 회사, 지역, 산업, 직군.
링크 테이블	공고와 엔티티의 다대다 관계를 잇는 표. posting_tech 등.
taxonomy	여러 표기를 표준 기술 이름으로 모으는 사전.
canonical / alias	표준 이름 / 그 별칭.
co-occurrence	두 기술이 한 공고에 함께 요구되는 것. 동시 요구.
mart.db	수집과 정제가 끝난 정본 SQLite 파일.
Postgres	서비스가 쓰는 데이터베이스. RAG가 도는 곳.

RAG 기초. 검색으로 근거를 만든 다음에 답한다

1. 개요

이 문서는 RAG(Retrieval-Augmented Generation)를 처음 접하는 완전 초보자를 위한 입문 문서다. 언어모델이 무엇이고 왜 그럴듯한 거짓말을 지어내는지부터 시작해서, 검색을 붙여 근거를 주는 것이 왜 RAG라는 이름으로 불리는지, 모델을 다시 학습시키는 파인튜닝과는 어떻게 다른지, 그리고 언제 RAG를 쓰는 것이 유리한지를 순서대로 다룬다. 마지막에는 이 프로젝트에서 RAG가 차지하는 위치를 정리한다.

이 문서는 공고, 점수화, 서비스 데이터 같은 프로젝트 용어를 설명 없이 쓰는데, 그 바닥은 [00-orientation.md](#)에 이미 정리해두었다. 프로젝트와 데이터 구조가 낯설다면 그 문서를 먼저 읽는다.

2. LLM의 정의와 할루시네이션 원인

RAG를 이해하려면 먼저 그 재료가 되는 대규모 언어모델(LLM, Large Language Model)이 어떻게 동작하는지 알아야 한다. LLM은 인터넷에 있는 방대한 양의 글을 학습해서, 어떤 문장이 주어졌을 때 그다음에 올 단어 조각인 토큰을 확률적으로 예측하도록 훈련된 프로그램이다. 질문에 답할 때도 모델은 사실 여부를 확인하는 별도의 검증 절차를 거치지 않고, 학습 과정에서 익힌 통계적 패턴을 바탕으로 가장 자연스럽게 이어질 것 같은 단어를 하나씩 골라 이어붙여 문장을 완성한다[1].

이 방식은 대부분의 상황에서 매끄럽고 사람이 쓴 것 같은 문장을 만들어내지만, 부작용도 함께 따라온다. 모델의 학습 목표가 처음부터 사실을 정확히 진술하는 것이 아니라 다음 토큰을 그럴듯하게 잇는 것이었기 때문에, 정확성과 그럴듯함이 어긋나는 순간에는 그럴듯함이 이긴다. 그 결과 모델이 답을 실제로 알지 못하는 질문에서도 마치 알고 있는 것처럼 자연스러운 문장을 지어내는 현상이 나타나는데, 이를 할루시네이션(hallucination)이라고 부른다[1]. 여기에 더해 사람이 선호하는 답을 학습시키는 후속 훈련 과정에서 "모른다"고 답하는 모델보다 확신에 차서 답하는 모델이 더 좋은 평가를 받는 경향이 있고, 이 경향이 모델이 모르는 내용조차 자신 있는 어조로 지어내도록 강화하는 요인이 되기도 한다[1].

정리하면 LLM은 기억을 조회하는 기계가 아니라 다음 단어를 예측하는 기계이고, 이 근본적인 동작 방식 때문에 모르는 것을 모른다고 인정하기보다 그럴듯한 답을 지어내는 쪽으로 기울어져 있다. RAG는 바로 이 지점을 겨냥해서 설계된 방식이다.

3. RAG의 정의

RAG는 "Retrieval-Augmented Generation", 즉 검색으로 보강한 생성을 의미하며, 대규모 언어모델의 답변이 학습 데이터가 아닌 신뢰할 수 있는 외부 지식 기반을 참조하도록 최적화하는 절차다[2]. 이 정의를 처음 제시한 2020년 논문은 언어모델의 지식을 두 종류로 나누어 설명하는데, 모델의 가중치 안에 저장되어 학습이 끝나면 고정되는 지식을 파라메트릭(parametric) 메모리라 부르고, 모델 바깥의 문서 저장소에 있어서 언제나 갱신할 수 있는 지식을 논파라메트릭(non-parametric) 메모리라 부른다[3][4]. RAG는 이 두 메모리를 결합해서, 모델의 생성 능력은 그대로 유지한 채 답을 만드는 시점마다 최신 정보를 외부에서 조회해 덧붙이는 구조를 만든다.

이 이름은 그대로 동작 순서를 가리킨다.

- Retrieve(검색):** 사용자의 질문과 관련된 자료를 보유 데이터(DB, 문서, 그래프 등)에서 찾아온다.
- Augment(보강):** 찾아온 자료를 질문과 함께 프롬프트에 결합해서, 모델이 참고할 근거를 명시적으로 만들어준다.
- Generate(생성):** 보강된 프롬프트를 언어모델에 전달해서, 그 근거를 바탕으로 답을 생성한다.

핵심은 이 순서에 있다. 언어모델은 먼저 생각하고 답하는 것이 아니라 먼저 찾고, 찾은 것 안에서만 답한다. 다시 말해 언어모델이 파라미터 지식만으로 답을 즉석에서 구성하게 두지 않고, 검색으로 확보한 근거 문서 범위 안에서만 답을 구성하도록 강제하는

방식이다.

오픈북 시험에 비유할 수 있다. RAG가 없는 언어모델은 책을 덮고 기억나는 대로 쓰는 학생이고, RAG를 쓰는 언어모델은 문제를 보자마자 관련 페이지를 펼쳐 그 내용만 인용해서 쓰는 학생이다. 후자는 책에 없는 내용을 답으로 쓸 이유가 없고, 책이 바뀌면 곧바로 그 내용을 답에 반영할 수 있다.

아래 그림은 이 차이를 흐름으로 보여준다. 위쪽은 질문을 검증 없이 곧바로 모델에 넣는 방식이고, 아래쪽은 질문과 답 사이에 검색과 근거 결합 단계를 끼워 넣은 RAG의 흐름이다.

그대로 호출



Augment → Generate



그림 1. AI API를 그대로 호출하는 방식은 질문을 바로 LLM에 넣어 근거 없는 추측성 답을 얻는다. RAG는 질문과 답변 사이에 검색과 근거 결합 단계를 넣어, LLM이 실제 문서를 인용해 답하도록 만든다.

이 흐름을 아주 단순화한 의사코드로 적으면 다음과 같다. 실제 구현은 검색 대상과 프롬프트 구성 방식에 따라 훨씬 복잡해지지만, 뼈대는 항상 이 세 단계를 벗어나지 않는다.

```
def rag_answer(question):  
    documents = retrieve(question, top_k=5) # 검색: 관련 문서를 찾아온다  
    prompt = augment(question, documents) # 보강: 질문과 근거를 결합한다  
    answer = generate(prompt) # 생성: 근거를 바탕으로 답한다  
    return answer
```

4. RAG vs "AI API만 떼다 쓰기"

가장 단순한 형태의 AI 기능은 사용자의 질문을 그대로 언어모델 API에 전달하고 반환된 답을 그대로 노출하는 방식인데, 이는 RAG가 아니며 앞서 설명한 문제를 그대로 안고 있다.

- 언어모델은 서비스 고유 데이터(광고, 통계, 사용자 정보)를 전혀 알지 못하고, 학습 시점에 습득한 일반 지식만 보유한다.
- 그 결과 서비스 관련 질문에 대해 모델이 답을 알지 못함에도 그럴듯하게 지어낼 위험이 크다.
- 데이터가 매일 갱신되어도(새 광고 게시, 통계 갱신) 모델의 지식은 고정되어 있어서, 재학습 없이는 최신 정보를 반영할 방법이 없다.

RAG는 이 문제를 모델을 재학습시키는 대신, 질문이 들어올 때마다 최신 데이터를 검색해 모델에 전달하는 방식으로 해결한다. 모델의 파라미터(가중치)는 고정된 채 답을 생성하는 시점에 필요한 근거만 주입하는 구조이며, 그래서 데이터가 변경되어도 재학습이 필요 없고 모델이 원래 알지 못하는 서비스 데이터에 대해서도 정확한 답변이 가능해진다.

정리하면, "AI API만 떼다 쓰기"는 질문을 모델에 그대로 전달하는 방식이고 RAG는 질문에 답하기 전에 데이터에서 근거를 먼저 확보해 모델에 전달하는 방식인데, 이 한 단계 차이가 답변의 신뢰도를 결정한다.

5. RAG vs 파인튜닝의 차이

RAG와 자주 비교되는 또 다른 접근이 파인튜닝(fine-tuning)이다. 파인튜닝은 특정 도메인의 데이터로 모델을 추가 학습시켜서 모델 내부의 가중치 자체를 바꾸는 방식이다. 반면 RAG는 모델의 가중치를 전혀 건드리지 않고, 답을 생성하는 순간에 참고할 자료를 프롬프트에 얹어주는 방식이다. 비유하자면 파인튜닝은 학생을 다시 가르쳐서 사고방식과 말투 자체를 바꾸는 것이고, RAG는 시험장에 참고 자료를 들고 들어가게 해주는 것이다[5].

이 차이 때문에 두 방식은 잘하는 영역이 다르다.

- **RAG가 유리한 경우:** 정보가 자주 바뀌거나, 출처를 밝혀야 하거나, 사용자마다 접근 가능한 데이터가 다른 경우다. 새로운 문서를 색인에 추가하기만 하면 곧바로 반영되고, 답변에 근거 문서를 인용할 수 있어서 검증이 쉽다[5].
- **파인튜닝이 유리한 경우:** 모델이 답하는 방식이나 말투, 형식을 일관되게 바꾸고 싶거나, 분류나 정형 추출처럼 좁고 반복적인 작업을 대량으로 처리해야 하는 경우다. 다만 품질이 좋은 학습 데이터를 충분히 확보해야 하고, 도메인 지식이 바뀔 때마다 다시 학습시켜야 하는 비용이 든다[5].

실무에서는 이 둘을 배타적으로 고르지 않고 함께 쓰는 경우가 많다. 답변의 말투나 형식은 파인튜닝으로 다듬고, 답변에 들어갈 사실은 RAG로 그때그때 조회해서 근거를 붙이는 식이다[5]. 이 프로젝트는 서비스 데이터가 매일 갱신되고 답변마다 근거 인용이 중요하다는 점에서 RAG 쪽에 무게가 실리는 상황이며, 그래서 파인튜닝 대신 RAG를 핵심 접근으로 채택했다.

6. 필요성: 할루시네이션과 최신성

지금까지 다른 내용을 종합하면, RAG가 해결하는 문제는 크게 두 가지로 압축된다.

할루시네이션(hallucination). 앞서 설명했듯 언어모델은 모르는 질문에도 "모른다"고 답하기보다 그럴듯한 문장을 생성하는 경향을 보인다. 특히 숫자나 통계처럼 정확성이 중요한 질문에서는 이 경향이 위험 요소가 되는데, "이 지역 채용 공고가 몇 건인가"라는 질문에 실제 DB를 조회하지 않고 답하면 그럴듯하지만 틀린 숫자가 나올 수 있기 때문이다. RAG는 답변을 생성하기 전에 실제 데이터 검색을 강제함으로써 이런 근거 없는 답변을 원천적으로 줄인다.

최신성(freshness). 언어모델은 학습이 종료된 시점의 지식만 보유하기 때문에, 채용 공고가 매일 게시되고 통계가 지속적으로 갱신되는 도메인에서는 모델의 성능과 무관하게 어제 새로 올라온 공고를 알 방법이 없다. RAG는 질문이 들어올 때마다 그 시점의 최신 데이터를 검색하므로, 모델을 매번 재학습시키지 않고도 항상 최신 정보로 답할 수 있다.

이 두 문제를 종합하면 다음 원칙이 도출된다.

답은 검색으로 찾은 근거 안에서만 만들어져야 하며, 근거에 없는 값은 생성하지 않는다.

이는 이 프로젝트 전체를 관통하는 정직성 원칙이기도 한데, 개수, 순위, 비율 같은 정량적인 질문은 벡터 검색이나 그래프가 아니라 반드시 결정론적인 SQL로 답하도록 하고, 검색으로 충분한 근거를 확보하지 못하면 "모른다" 또는 "데이터가 부족하다"고 정직하게 답하도록 한다. 이 원칙이 실제로 어떻게 구현되는지는 02 부터 04 문서에서 반복적으로 다룬다.

7. RAG의 다양한 방식

여기까지는 RAG를 가장 단순한 형태로만 설명했지만, 실제로는 검색과 생성을 어떻게 조합하느냐에 따라 여러 방식으로 나뉜다. 질문을 임베딩해서 벡터 데이터베이스에서 상위 몇 개 문서만 가져와 곧바로 답을 생성하는 가장 기본적인 형태를 Naive RAG라 부르고, 여기에 데이터 전처리 개선, 검증 단계, 반복 검색 같은 장치를 더한 형태를 Advanced RAG 또는 Modular RAG라 부른다[6]. 더 나아가 에이전트가 질문의 성격에 맞는 도구를 스스로 고르고 근거가 부족하면 재검색하는 Agentic RAG, 문서 사이의 관계를 그래프로 미리 구축해두고 그 그래프를 순회하며 답하는 Graph RAG도 있다. 이 각각의 방식이 정확히 무엇이고 언제 쓰는 것이 적합한지는 03-rag-types.md 에서 자세히 다룬다.

8. 우리 프로젝트에서의 위치

이 프로젝트에서 AI/RAG 기능은 부가기능(bonus feature)이다. 채용 정보를 점수화해서 보여주는 핵심 기능이 별도로 존재하고 RAG 챗봇은 그 위에 얹히는 부가 기능이지만, 그렇다고 이 부가기능을 질문을 벡터 검색 한 번 돌려 상위 결과를 모델에 붙이는 수준으로 나이브하게 구현하지 않고 진지하게 설계했다.

이유는 두 가지다.

1. **"AI API만 떼다 쓴 게 아니다"를 증명한다.** 단순히 API를 호출해 그럴듯한 답을 내는 구현은 반나절이면 가능하지만, 이 시스템은 질문의 성격에 따라 SQL, 벡터 검색, 지식그래프 중 적절한 도구를 선택하도록 만들었다. 답이 충분한 근거를 갖췄는지 스스로 검증한 뒤 부족하면 재검색하는 구조인 Agentic RAG와, 채용 데이터 내 관계(기술 동시 요구, 회사와 지역의 관계 등)를 그래프로 순회해 답하는 구조인 Graph RAG를 함께 사용하는데, 이 설계는 [03-rag-types.md](#) 와 [04-our-architecture.md](#) 에서 자세히 다룬다.
2. **부가기능에서도 정직성을 지킨다.** 부가기능이라는 이유로 근거 없는 답변을 허용하면 서비스 전체의 신뢰가 훼손되기 때문에, 이 RAG 시스템도 근거 인용 필수, 정량은 SQL, 모르면 모른다고 답하기라는 본 기능과 동일한 원칙을 지키도록 설계했다.

이어지는 문서들은 이 원칙이 구체적으로 어떤 기술(임베딩, 벡터 DB, 지식그래프, 에이전트 파이프라인)로 구현되는지 순서대로 설명한다.

9. 참고 자료

1. [Hallucinations in LLMs Are Not a Bug in the Data](#)
2. [What is RAG? Retrieval-Augmented Generation AI Explained](#)
3. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)
4. [Retrieval-augmented generation](#)
5. [RAG vs. Fine-tuning vs. Prompt Engineering: The Complete Guide to AI Optimization](#)
6. [Retrieval-Augmented Generation for Large Language Models: A Survey](#)

임베딩과 벡터 DB, 의미를 숫자로 바꿔 비교하기

1. 개요

RAG의 검색 단계에서 가장 핵심적인 기술은 임베딩과 벡터 DB다. 이 둘이 있어야 "글자가 같은지"가 아니라 "뜻이 비슷한지"를 계산으로 비교하는 일이 가능해진다. 이 문서는 완전히 처음 이 개념을 접하는 사람을 기준으로 쓴다. 임베딩이 무엇인지에서 시작해 임베딩 모델이 어떻게 학습되는지, 두 벡터의 유사도를 어떻게 재는지, 벡터를 어디에 저장하고 어떻게 빠르게 찾는지, 마지막으로 청킹과 리랭킹까지 검색 파이프라인을 이루는 개념들을 순서대로 짚어간다. 임베딩의 대상이 되는 공고와 데이터 구조는 [00-orientation.md](#)에 정리해두었다.

2. 임베딩의 정의

컴퓨터는 원래 글자의 뜻을 모른다. 컴퓨터가 다룰 수 있는 것은 숫자뿐이고, 그래서 문장을 컴퓨터가 계산할 수 있는 형태로 바꾸는 작업이 필요한데, 이 변환의 결과물을 임베딩이라 한다. 임베딩은 문장이나 단어를 숫자로 이루어진 좌표, 즉 벡터로 바꾸는 작업이며 그 결과 나오는 좌표 자체도 임베딩이라 부른다. 좌표라고 하면 보통 x 와 y 두 개 숫자로 이루어진 평면 위의 점을 떠올리기 쉽지만, 임베딩은 보통 수백에서 수천 개의 숫자로 이루어진 좌표다. 이 좌표가 놓이는 공간을 고차원 공간이라 부르고, 좌표를 이루는 숫자의 개수를 차원이라 부른다.

이 변환에서 가장 중요한 약속은 하나다. 뜻이 비슷한 문장은 좌표도 서로 가깝게, 뜻이 다른 문장은 좌표도 서로 멀게 배치한다는 약속이다. 이 약속이 지켜지면 "이 문장과 저 문장의 뜻이 비슷한가"라는 어려운 질문이 "이 좌표와 저 좌표 사이의 거리가 가까운가"라는 단순한 계산 문제로 바뀐다.

세상의 모든 문장을 지도 위의 한 점으로 찍는다고 가정해본다. 뜻이 비슷한 문장들은 서로 가까운 위치에 찍히고 뜻이 다른 문장들은 먼 위치에 찍힌다. 예를 들어 "백엔드 개발자를 채용합니다"와 "서버 개발자를 모집합니다"는 겹치는 단어가 거의 없어도 지도 위에서는 이웃한 위치에 찍힌다. 반대로 "백엔드 개발자를 채용합니다"와 "오늘 점심 메뉴 추천"은 단어가 일부 겹치더라도 지도 위에서는 멀리 떨어진다.

이 성질이 왜 키워드 매칭보다 강력한지는 위 예시에서 바로 드러난다. 전통적인 키워드 검색은 질문과 문서에 같은 단어가 얼마나 들어 있는지를 센다. 그래서 "백엔드"라는 단어로 검색하면 "서버 개발자를 모집합니다"라는 공고는 단어가 하나도 겹치지 않으므로 찾아내지 못한다. 사람이 보기에는 두 표현이 거의 같은 뜻인데도, 글자 그대로만 비교하는 키워드 검색은 이 관계를 알아채지 못하는 것이다. 반면 임베딩은 학습 과정에서 이미 "백엔드"와 "서버"가 비슷한 맥락에서 쓰인다는 사실을 좌표 관계로 흡수해두었기 때문에, 겹치는 글자가 하나도 없어도 두 문장의 좌표를 가깝게 배치한다. 오타자나 동의어, 줄임말, 다른 어순으로 쓰인 문장도 마찬가지로 뜻이 비슷하면 좌표가 가깝게 나온다는 점이 임베딩 기반 검색의 근본적인 장점이다.

이 시스템은 임베딩 모델로 **BGE-M3**를 로컬에서 직접 구동한다. BGE-M3는 문장을 1024차원 벡터로 변환하는 다국어 임베딩 모델이며, 최대 8192토큰까지 입력을 처리할 수 있어 긴 문서도 통째로 다룰 수 있다 [7]. 사용자 하드웨어인 RTX 4060 8GB VRAM 안에서도 구동이 가능하다는 점이 이 모델을 고른 실무적인 이유다. API 호출 없이 로컬 GPU로 임베딩을 생성한다는 점이 핵심이며, 답변을 문장으로 만드는 생성 작업만 Claude API에 맡기고 검색에 필요한 임베딩은 자체 호스팅한다. 이는 "AI API만 떼다 쓴 게 아니다"를 뒷받침하는 근거 중 하나다.

3. 임베딩 학습 방식

임베딩 모델이 처음부터 "백엔드"와 "서버"가 비슷하다는 사실을 알고 태어나는 것은 아니다. 이 관계는 학습 과정에서 만들어지는데, 그 핵심 원리를 대조학습이라 부른다.

대조학습의 발상은 단순하다. 뜻이 비슷한 문장 두 개를 짝으로 묶은 데이터, 예를 들어 같은 채용 공고를 다르게 표현한 두 문장이나 질문과 그 질문에 맞는 답변 문장의 짝을 대량으로 모은다. 이런 짝을 양성 쌍이라 하고, 반대로 뜻이 다른 문장끼리 묶은 것을 음성 쌍이라 한다. 학습 중인 모델은 문장을 좌표로 변환한 다음, 양성 쌍의 좌표는 서로 끌어당겨 더 가깝게 만들고 음성 쌍의 좌표는 서로 밀어내어 더 멀게 만드는 방향으로 조금씩 조정된다. 이 과정을 수많은 문장 쌍에 걸쳐 반복하면, 모델은 점차 "이런 식으로 표현이 달라도 뜻이 같으면 가깝게 배치해야 하는구나"라는 규칙을 좌표 공간 안에 새겨 넣게 된다.

이 과정은 사진을 정리하는 일과 비슷하다. 처음에는 사진들이 아무 서랍에나 뒤섞여 있지만, "이 두 장은 같은 여행에서 찍은 사진이다"라는 정보가 주어질 때마다 그 두 장을 같은 서랍 쪽으로 옮기고, "이 두 장은 전혀 다른 날 찍은 사진이다"라는 정보가 주어질 때마다 서로 먼 서랍으로 떨어뜨려 놓는다. 이 작업을 수백만 장에 걸쳐 반복하면 결국 비슷한 사진들이 자연스럽게 같은 구역에 모이는 정리함이 완성된다.

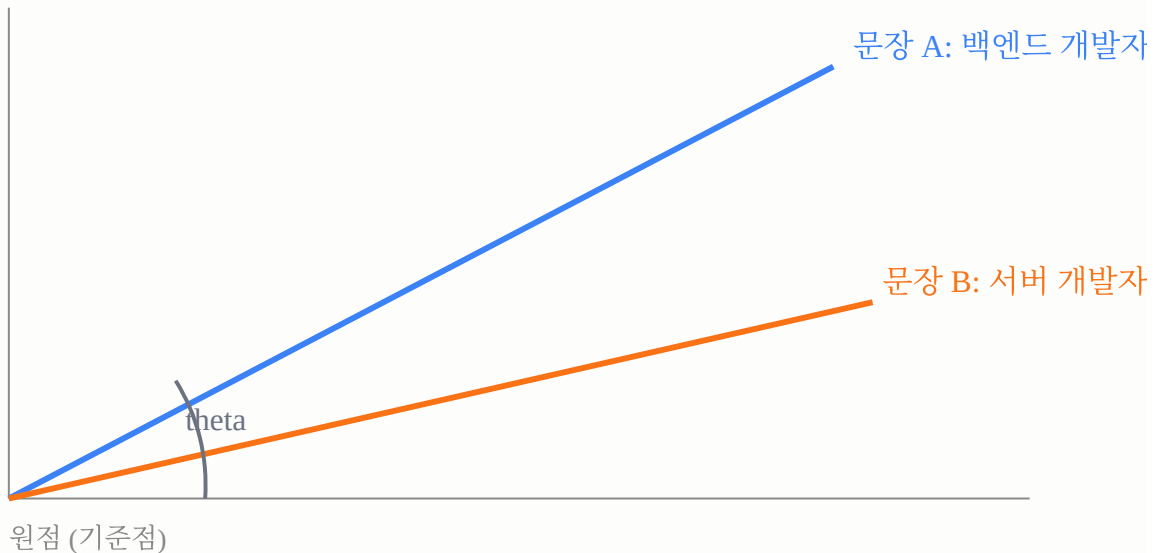
실제 학습에서는 한 번에 문장 하나씩이 아니라 여러 문장을 묶은 배치 단위로 처리하며, 배치 안에서 정답 짝은 아니지만 우연히 같이 뽑힌 다른 문장들을 음성 쌍으로 활용하는 방식이 흔히 쓰인다. sentence-transformers 계열 모델은 이런 대조학습 방식으로 문장 쌍의 코사인 유사도를 계산하고, 정답 짝의 유사도는 높이고 나머지 짝의 유사도는 낮추는 손실 함수로 학습을 진행한다 [8]. BGE-M3 같은 모델은 이 원리를 여러 언어와 여러 문서 길이에 걸쳐 대규모로 학습해, 언어가 달라도 뜻이 비슷하면 좌표가 가깝게 나오는 성질까지 갖추게 된다.

4. 두 벡터가 얼마나 비슷한지 재는 법: 코사인 유사도

문장을 좌표로 바꾸고 나면 두 좌표가 얼마나 가까운지를 숫자 하나로 표현할 방법이 필요하다. 임베딩 검색에서 가장 널리 쓰이는 방법이 코사인 유사도다.

코사인 유사도는 거리 대신 방향을 본다는 점이 특징이다. 두 벡터를 원점에서 출발하는 화살표로 그렸을 때, 두 화살표가 벌어진 각도가 작을수록 두 벡터는 같은 방향을 가리키고, 각도가 클수록 서로 다른 방향을 가리킨다. 코사인 유사도는 바로 이 각도의 코사인 값이다.

각도가 작을수록 코사인 유사도는 1에 가까워진다



두 벡터가 완전히 같은 방향이면 각도는 0도이고 코사인 유사도는 1이 되어 뜻이 매우 유사하다고 본다. 두 벡터가 직각을 이루는 90도가 되면 코사인 유사도는 0이 되어 서로 관련이 없다고 보고, 두 벡터가 정반대 방향인 180도가 되면 코사인 유사도는 -1이 되어 정반대 의미로 판단한다. 수식으로 쓰면 벡터 A와 B의 코사인 유사도는 다음과 같다.

$$\cos(\theta) = (A \text{ dot } B) / (\|A\| \times \|B\|)$$

여기서 $\mathbf{A} \cdot \mathbf{B}$ 는 두 벡터의 내적으로, 같은 위치의 숫자끼리 곱한 다음 전부 더한 값이다. $\|\mathbf{A}\|$ 와 $\|\mathbf{B}\|$ 는 각 벡터의 길이로, 벡터를 이루는 숫자들을 각각 제곱해 더한 다음 제곱근을 씌운 값이다. 분모에서 두 벡터의 길이를 곱한 값으로 나눠주기 때문에, 이 값은 벡터의 길이와는 무관하게 오직 방향, 즉 각도만을 반영하는 값으로 정규화된다고 [5][6].

여기서 한 가지 실용적인 사실이 나온다. 만약 두 벡터를 미리 길이가 1이 되도록 정규화해두면, 즉 $\|\mathbf{A}\|$ 와 $\|\mathbf{B}\|$ 가 둘 다 1이면, 분모가 사라지고 코사인 유사도는 그냥 두 벡터의 내적과 같아진다. 벡터 데이터베이스들이 내부적으로 벡터를 정규화해서 저장하거나 내적 연산으로 코사인 유사도를 대신 계산하는 이유가 여기에 있다. 내적은 곱하고 더하기만 하면 되는 가장 가벼운 연산이라서, 정규화만 미리 해두면 코사인 유사도 계산을 내적 계산으로 바꿔 속도를 아낄 수 있다 [5].

코사인 유사도와 자주 비교되는 다른 척도로 유클리드 거리가 있다. 유클리드 거리는 두 점을 잇는 직선의 길이, 즉 우리가 흔히 생각하는 "실제 거리"를 그대로 잴 수 있다. 코사인 유사도는 벡터의 방향만 보고 길이는 무시하지만, 유클리드 거리는 방향뿐 아니라 벡터의 길이, 즉 크기 차이까지 그대로 반영한다. 그래서 길이가 크게 다른 두 벡터라도 방향이 같으면 코사인 유사도는 1에 가깝게 나오지만 유클리드 거리는 크게 나올 수 있다. 문장 임베딩에서는 문장의 길이나 표현의 강도보다 의미의 방향이 중요하기 때문에 코사인 유사도를 기본값으로 쓰는 경우가 많고, 반면 벡터의 절대적인 크기 자체가 의미를 갖는 경우에는 유클리드 거리가 더 알맞다 [9]. 실제 검색에서는 사용자 질문을 임베딩해 좌표로 만든 다음, 미리 임베딩해둔 문서들의 좌표와 코사인 유사도를 전부 계산하고, 유사도가 가장 높은 상위 k개, 즉 top-k를 근거 후보로 확보한다.

간단한 예로, sentence-transformers 라이브러리로 문장을 벡터로 바꾸고 유사도를 계산하는 코드는 다음과 같다.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

sentences = [
    "백엔드 개발자를 채용합니다.",
    "서버 개발자를 모집합니다.",
    "오늘 점심 메뉴 추천 부탁드립니다.",
]

embeddings = model.encode(sentences)
print(embeddings.shape) # (3, 384) : 문장 3개, 각각 384차원 벡터

similarities = model.similarity(embeddings, embeddings)
print(similarities)
# 대각선은 자기 자신과의 유사도라 항상 1에 가깝고,
# 1번과 2번 문장(백엔드/서버)의 유사도가 3번 문장(점심 메뉴)보다 훨씬 높게 나온다
```

이 코드에서 `model.similarity` 는 내부적으로 코사인 유사도를 계산해준다. 실제로 위 모델로 비슷한 예시 문장 세 개를 인코딩하면, 관련 있는 문장 쌍끼리는 유사도가 0.6 이상으로 높게 나오고 관련 없는 문장과의 유사도는 0.1 근처로 낮게 나오는 패턴을 확인할 수 있다 [10].

5. 차원이 의미하는 것

앞서 임베딩은 수백에서 수천 개 숫자로 이루어진 좌표라고 했는데, 이 숫자의 개수를 차원이라 부른다. BGE-M3는 문장 하나를 1024개의 숫자로 이루어진 벡터로 바꾸므로 1024차원 임베딩을 만든다고 말한다 [7].

차원이 왜 여러 개나 필요한지는 다음과 같이 이해할 수 있다. 좌표가 2차원, 즉 숫자 두 개짜리라면 표현할 수 있는 의미의 방향이 평면 위 각도 하나로만 제한된다. 그런데 문장의 의미는 주제, 어투, 시제, 긍정과 부정, 전문 분야, 문장 길이 등 매우 다양한 성질이 동시에 얹혀 있다. 차원이 많을수록 이런 성질 각각을 서로 다른 방향으로 나누어 담을 여유가 생기고, 그만큼 더 섬세하게 의미를 구별할 수 있다.

다만 차원이 늘어난다고 무조건 좋은 것은 아니다. 차원이 커질수록 벡터 하나를 저장하는 데 필요한 용량이 늘어나고, 벡터 사이의 거리를 계산하는 데 드는 연산량도 함께 늘어난다. 또한 검색 속도에 직접 영향을 주는 인덱스의 크기도 차원에 비례해 커진다. 그래서 임베딩 모델을 고를 때는 의미를 얼마나 세밀하게 담아내는지와, 그 벡터를 저장하고 검색하는 데 드는 비용 사이에서 균형을 잡아야 한다. 1024차원은 다국어 문서를 폭넓게 다루면서도 실무에서 다룰 만한 크기로 널리 쓰이는 선택 중 하나다.

6. 벡터 DB와 pgvector

문서 수가 적으면 코사인 유사도를 문서 전체에 대해 전수 계산해도 되지만, 문서가 수만에서 수백만 건에 이르면 매번 전수 비교하는 방식은 비효율적이므로, 이를 빠르게 처리하는 전용 저장소로 벡터 DB를 쓴다. 벡터 DB는 임베딩 좌표를 저장해두고 "이 좌표와 가장 가까운 좌표 k개를 찾아달라"는 질의를 효율적으로 처리하는 역할을 한다.

벡터 DB 전용 제품인 Pinecone, Milvus, Weaviate 등을 신규 도입하는 방안도 있었으나, 이 시스템은 pgvector를 채택했다. pgvector는 기존에 사용 중인 PostgreSQL에 벡터 타입과 벡터 검색 기능을 추가하는 확장으로, 별도 인프라를 새로 구축하지 않고도 기존 Postgres 테이블 안에 벡터 컬럼을 두어 SQL로 벡터 검색까지 함께 처리할 수 있다 [1]. 특히 이 시스템은 정량 질문을 SQL로, 의미 질문을 벡터로 라우팅하는 구조이므로, 두 경로를 동일한 Postgres 안에서 처리할 수 있다는 점이 실용적인 이점으로 작용했다.

pgvector를 쓰는 방식은 다음과 같다. 먼저 확장을 설치하고, 테이블에 벡터 타입 컬럼을 만든 다음, 코사인 거리를 뜻하는 `<=>` 연산자로 가장 가까운 벡터를 찾는다 [1].

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE job_postings (
  id bigserial PRIMARY KEY,
  title text NOT NULL,
  embedding vector(1024)
);

-- 질문 벡터(:query_embedding)와 가장 가까운 공고 5건을 코사인 거리 기준으로 조회
SELECT id, title, embedding <=> :query_embedding AS distance
FROM job_postings
ORDER BY embedding <=> :query_embedding
LIMIT 5;
```

여기서 `<=>` 연산자는 코사인 거리를 반환하는데, 코사인 거리는 코사인 유사도와 반대로 값이 작을수록 더 비슷하다는 뜻이다. 실제로 코사인 거리는 `1 - 코사인 유사도`로 계산되므로, 유사도가 1이면 거리는 0이 되고 유사도가 0이면 거리는 1이 된다. 그래서 검색 쿼리에서는 거리가 작은 순서, 즉 오름차순으로 정렬해 가장 가까운 결과를 먼저 가져온다 [1].

7. 전수 검색의 한계와 근사 최근접 이웃

앞의 SQL 예시처럼 `ORDER BY embedding <=> :query_embedding`을 인덱스 없이 실행하면, Postgres는 테이블에 있는 모든 행의 벡터와 질문 벡터 사이의 거리를 하나하나 계산한다. 이 방식을 전수 검색이라 부르는데, 문서 수가 늘어나면 계산량이 그 수에 정확히 비례해서 늘어난다. 문서가 천 건이면 천 번 계산하면 되지만 문서가 백만 건이면 백만 번을 계산해야 하고, 벡터 하나의 차원이 클수록 계산 한 번의 비용도 커지므로 대규모 데이터에서는 매 질의마다 이 비용을 감당하기 어려워진다.

이 문제를 해결하기 위해 쓰는 접근이 근사 최근접 이웃, 줄여서 ANN이다. ANN은 "완벽하게 정확한 최근접 이웃을 찾겠다"는 목표를 조금 내려놓고, "매번 조금씩 부정확해도 되니 훨씬 빠르게 찾겠다"는 쪽으로 타협한 방법론이다 [3]. 실제로는 최상위 몇 개 후보 중 진짜 최근접 이웃이 누락되는 경우가 드물게 있지만, 그 대신 검색 속도가 전수 검색과는 비교할 수 없이 빨라지기 때문에, 실무에서는 이 근사치로도 충분한 경우가 대부분이다. pgvector가 지원하는 대표적인 ANN 인덱스가 HNSW다.

8. HNSW 인덱스

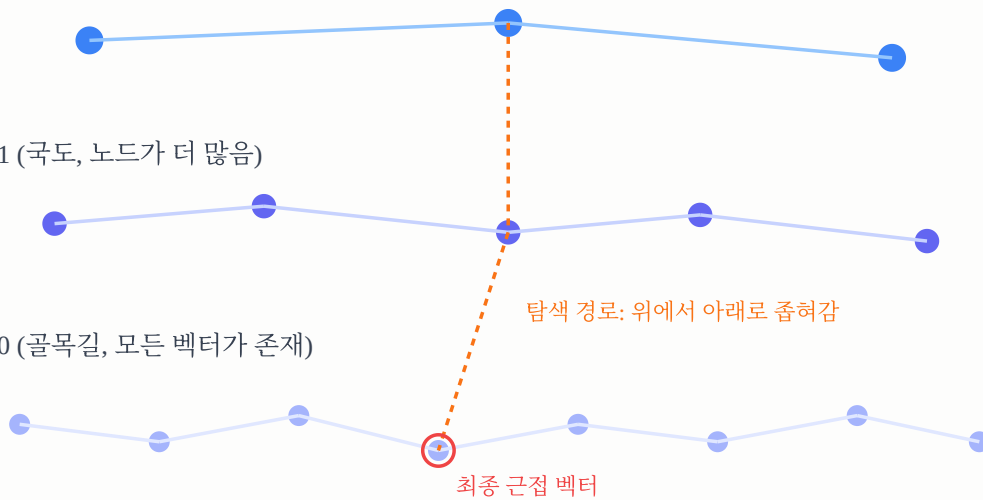
HNSW는 Hierarchical Navigable Small World의 줄임말로, 계층을 이룬 그래프 구조를 이용해 벡터를 빠르게 탐색하는 인덱스 방식이다. 2016년에 발표된 논문에서 제안된 이 구조는, 근접한 벡터들을 그래프의 노드와 엣지로 연결해두고 이 그래프를 여러 층으로 쌓아 올려서 탐색 범위를 단계적으로 좁혀가는 방식으로 동작한다 [4].

HNSW는 고속도로에서 국도를 거쳐 골목길로 이어지는 다층 도로망에 비유할 수 있다. 최상위 레이어에는 들판처럼 고속도로만 있어서 목적지 근처까지 몇 번의 큰 점프로 빠르게 이동하고, 하위 레이어로 내려갈수록 촘촘한 골목길이 나타나면서 여기서 목적지를 정밀하게 좁혀간다. 이 구조 덕분에 전체 데이터를 하나하나 비교하지 않아도 되는데, 전수 비교는 데이터가 늘어날수록 선형으로 느려지는 반면 HNSW는 목적지 근처의 후보만 빠르게 좁혀 찾을 수 있다.

레이어 2 (고속도로, 노드가 가장 적음)

레이어 1 (국도, 노드가 더 많음)

레이어 0 (골목길, 모든 벡터가 존재)



검색은 항상 가장 위쪽, 노드 수가 가장 적은 레이어에서 시작한다. 이 레이어에서 질문 벡터와 가까운 노드를 대략적으로 찾은 다음, 그 노드를 입구로 삼아 한 레이어 아래로 내려간다. 아래 레이어는 노드 수가 더 많고 연결이 더 촘촘하므로 여기서 다시 더 정밀하게 가까운 노드를 찾고, 이 과정을 가장 아래 레이어까지 반복한다. 위쪽 레이어에서 이미 대략적인 위치를 좁혀두었기 때문에, 아래로 내려갈 때마다 비교해야 하는 후보의 범위가 크게 줄어든다. 이 덕분에 전체 데이터 규모와 무관하게 각 질의가 거쳐야 하는 비교 횟수를 로그 수준으로 억제할 수 있다 [3][4].

HNSW 인덱스에는 이 그래프의 모양과 탐색 폭을 조절하는 파라미터가 있다.

파라미터	의미	pgvector 기본값
<code>m</code>	각 노드가 한 레이어 안에서 갖는 최대 연결 수	16
<code>ef_construction</code>	인덱스를 만들 때 각 노드의 연결 후보를 얼마나 넓게 탐색할지	64
<code>ef_search</code>	검색할 때 후보를 얼마나 넓게 탐색할지	40

`m` 이 커지면 노드마다 더 많은 이웃과 연결되므로 그래프가 촘촘해져 검색 정확도, 즉 재현율이 올라가지만, 인덱스가 차지하는 메모리와 구축 시간도 함께 늘어난다. `ef_construction` 은 인덱스를 만드는 시점에 각 노드를 그래프에 끼워 넣을 때 얼마나 많은 후보를 검토해서 연결선을 정할지를 정하는데, 이 값이 크면 더 좋은 품질의 그래프가 만들어지지만 인덱스 구축 자체가 오래 걸린다. `ef_search` 는 실제 검색 시점에 몇 개의 후보를 훑어보며 다음 레이어로 내려갈지를 정하는데, 이 값이 크면 더 정확한 결과를 얻지만 검색 한 번에 걸리는 시간도 늘어난다. 세 값 모두 정확도와 속도, 그리고 자원 사용량 사이의 트레이드오프를 조절하는 손잡이라고 볼 수 있다 [1][2][11].

```
-- HNSW 인덱스 생성 (m, ef_construction은 빌드 시점 파라미터)
CREATE INDEX ON job_postings USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- 검색 시점에 탐색 폭을 조절 (세션 단위로 설정)
SET hnsw.ef_search = 100;
```

실제로 채택한 값과 그 근거는 구현기 [rag-build-journal](#) 의 "벡터 인덱스 실전기"에서 실측과 함께 기록한다.

9. 청킹

채용 공고 상세 페이지 전체처럼 문서 하나가 매우 길면, 통째로 임베딩하기보다 적당한 크기로 분할해 여러 개의 임베딩으로 나누는 것이 일반적이는데, 이 분할 작업을 청킹이라 한다.

청킹이 필요한 이유는 분명하다. 문서 전체를 하나의 벡터로 뭉뚱그리면 문서 안의 여러 주제가 서로 희석되어 검색 정확도가 떨어지고, 반대로 지나치게 잘게 자르면 문맥이 끊겨 의미가 모호해질 수 있다. 그래서 문단 단위나 의미 단위처럼 적절한 단위로 분할하는 것이 관건이다.

이 시스템에서 중요한 원칙은 원본 채용 공고 텍스트를 통째로 임베딩하지 않는다는 점이다. 대신 요약, 통계 리포트, 지식그래프의 커뮤니티 리포트처럼 이미 한 차례 정제된 단위를 임베딩함으로써 노이즈를 줄이고, 검색 결과가 공고 원문 조각이 아니라 의미 있게 요약된 근거가 되도록 한다.

10. 리랭커

코사인 유사도 기반 top-k 벡터 검색은 빠르지만 완벽하지 않은데, 좌표가 가깝다고 해서 질문과의 관련성 순서가 항상 정확히 정렬되어 있는 것은 아니기 때문이다. 그래서 벡터 검색으로 일단 후보를 넉넉히 확보한 다음, 더 정교한 모델로 해당 후보만 다시 순위를 매기는 단계를 두는데, 이를 리랭커라 한다.

벡터 검색은 도서관에서 주제와 관련 있어 보이는 책 50권을 서가에서 빠르게 뽑아오는 사서에 해당하고, 리랭커는 뽑혀온 50권을 직접 한 장씩 넘겨보며 질문에 실제로 부합하는 순서로 재정렬하는 전문가에 해당한다. 리랭커는 후보 수가 수십 개 수준으로 적으므로 더 무겁고 정교한 모델을 사용해도 무방하다.

이 시스템은 리랭커로 bge-reranker를 사용하며, BGE-M3와 마찬가지로 로컬 GPU인 RTX 4060 8GB에서 자체 호스팅한다. 벡터 검색으로 top-k 후보를 넉넉히 확보한 다음 bge-reranker로 최종 순위를 재산정하여, 관련성이 높은 근거만 답변 생성 단계로 전달한다.

11. 참고 자료

1. [pgvector README](#)
2. [Understanding vector search and HNSW index with pgvector](#)
3. [Hierarchical navigable small world - Wikipedia](#)
4. [Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs \(arXiv\)](#)
5. [Cosine similarity = dot product for normalized vectors](#)
6. [Vector Similarity Explained - Pinecone](#)
7. [BAAI/bge-m3 - Hugging Face](#)

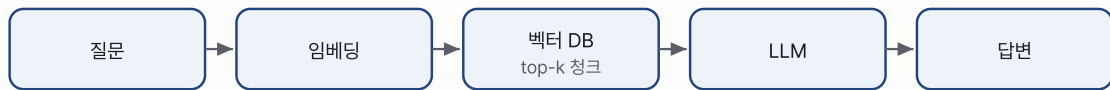
8. [Unsupervised Learning - Sentence Transformers documentation](#)
9. [Euclidean Distance vs Cosine Similarity - Baeldung](#)
10. [Quickstart - Sentence Transformers documentation](#)
11. [HNSW Indexes with Postgres and pgvector - Crunchy Data](#)

RAG의 종류: Naive부터 Agentic, Graph까지

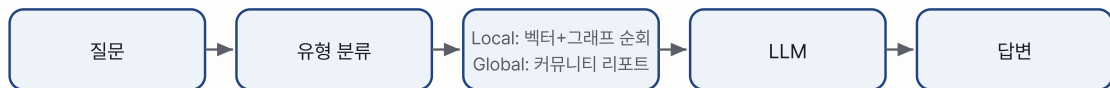
1. 개요

RAG는 하나의 고정된 방식이 아니라, 검색과 생성을 어떻게 조합하느냐에 따라 여러 세대로 발전해온 기술이다. 학계에서는 이 발전 과정을 크게 Naive RAG, Advanced RAG, Modular RAG 세 단계로 정리하는데, Naive RAG는 검색 한 번 뒤에 곧바로 답을 생성하는 가장 단순한 형태이고, Advanced RAG는 여기에 데이터 전처리 개선과 반복 검색 같은 장치를 더한 형태이며, Modular RAG는 라우팅과 여러 기능 모듈을 자유롭게 갈아 끼울 수 있는 유연한 형태다[1]. 이 문서는 이 흐름을 실무에서 가장 많이 부딪히는 세 가지 이름인 Naive, Agentic, Graph로 나누어 다루고, 마지막에는 이를 하나의 비교표로 정리한다. Agentic과 Graph는 각각 Modular RAG가 취할 수 있는 대표적인 형태라고 볼 수 있다. 공고 같은 프로젝트 용어는 [00-orientation.md](#) 에서 설명하며, 아래 그림은 세 방식의 파이프라인이 서로 어떻게 다른지 보여준다.

Naive RAG



Graph RAG



Agentic RAG



그림 1. 세 가지 RAG 파이프라인 비교. Naive는 벡터 검색 1회로 끝나고, Graph는 질문 유형에 따라 지역과 전역 검색을 나누며, Agentic은 도구 선택과 검증, 재검색 루프를 갖는다. 구조 비교의 뼈대는 ByteByteGo의 정리[2]를 참고해 자체 작도했다.

2. Naive RAG

Naive RAG는 가장 기본적인 형태이며, 흐름이 단순하다. 사용자 질문을 그대로 임베딩한 뒤 벡터 DB에서 코사인 유사도 기준 top-k 문서를 가져오고, 가져온 문서를 프롬프트에 그대로 붙여 언어모델에 "이 문서들을 참고해 답하라"고 요청하면 모델이 답을 한 번에 생성하는 것으로 흐름이 끝난다. 학계에서는 이 두 단계를 검색과 읽기(retrieval-reading)로 부르기도 한다[1].

Naive RAG는 구현이 간단하고 빠르지만, 한계도 뚜렷하다.

- 도구가 하나뿐이다. 질문이 "지금 통계 몇 건인가" 같은 정량 질문이든 "React와 함께 쓰는 기술은 무엇인가" 같은 관계 질문이든 무조건 벡터 검색 한 가지로 처리하는데, 정량 질문에 벡터 검색을 쓰면 숫자가 부정확해질 위험이 크다. 벡터는 의미가 비슷한 텍스트를 찾을 뿐, 정확한 집계를 계산하지 않기 때문이다.
- 검증이 없다. 검색된 문서가 질문에 답하기 충분한지 확인하는 단계가 없어서, 근거가 부족해도 그대로 답을 생성한다. 잘못된 청크를 가져와도 이를 바로잡을 장치가 아예 없다는 뜻이다[3].

- 정밀도와 재현율이 흔들린다. 질문과 표현이 다르면 관련 문서를 놓치거나, 관련 없는 청크가 함께 섞여 들어와 모델을 헷갈리게 한다. 특히 관련 청크가 길게 이어진 근거 문치 가운데에 묻히면 모델이 그 부분을 그냥 무시해버리는 현상이 알려져 있는데, 이를 "lost in the middle"이라 부른다[3].
- 문서를 일정 길이로 잘라 저장하는 청킹(chunking) 과정에서 하나의 답이 두 청크에 걸쳐 나뉘어 있으면, 어느 쪽 청크를 가져와도 답의 절반만 담겨 있는 문제가 생긴다[3].
- 관계를 파악하지 못한다. 벡터 검색은 개별 문서 단위로 유사한 것을 찾을 뿐이어서, 엔티티 사이의 구조적 관계(회사의 소재 지역, 소속 산업군 등)를 순회해 답하지 못한다.

3. Agentic RAG

Agentic RAG는 Naive RAG에 판단하는 두뇌를 더한 방식으로, 검색을 한 번에 끝내지 않는다. 여기서 에이전트(agent)란 스스로 상황을 판단해서 어떤 행동을 할지 결정하고, 필요하면 도구를 사용하고, 그 결과를 보고 다음 행동을 다시 정하는 프로그램을 가리킨다[4]. Agentic RAG는 이 에이전트가 질문을 분석해 어떤 도구를 쓸지 스스로 선택하고, 검색 결과가 부족하면 재검색하는 루프를 수행한다는 점이 Naive RAG와 다르다.

일반적인 Agentic RAG 흐름은 다음과 같다.

1. 라우터 또는 플래너가 질문을 분석해 의도를 파악하고, 어떤 도구(SQL 조회, 벡터 검색, 그 외 API 등)를 쓸지 계획을 세운다. 질문을 분류해서 알맞은 처리 경로로 보내는 이 패턴을 라우팅(routing)이라 부른다[5].
2. 계획대로 도구를 실행하며, 필요하면 여러 도구를 동시에 실행한다.
3. 평가자(grader)가 확보된 근거가 질문에 답하기 충분한지 점검하고, 부족하면 도구나 검색어를 바꿔 재검색한다. 검색 결과를 평가해서 부족하면 스스로 고쳐 다시 검색하는 이 방식을 Self-RAG 또는 Corrective RAG라 부르며, 이 반성(reflection) 능력이 Agentic RAG를 Naive RAG와 구분 짓는 핵심 특징이다[6][5]. 보통 무한 루프를 막기 위해 재검색 횟수에 제한을 둔다.
4. 충분하다고 판단되면 합성(synthesis) 단계에서 근거를 인용해 최종 답을 생성한다.

이를 아주 단순화한 의사코드로 적으면 다음과 같다.

```
def agentic_rag_answer(question, max_retries=2):
    plan = route(question)          # 라우팅: 어떤 도구를 쓸지 결정
    for attempt in range(max_retries + 1):
        evidence = run_tools(plan)    # 도구 실행: SQL, 벡터, 그래프 등
        if is_sufficient(question, evidence): # 평가: 근거가 충분한가
            return synthesize(question, evidence)
        plan = rewrite(question, evidence) # 부족하면 계획을 다시 세운다
    return "충분한 근거를 찾지 못했다"
```

에이전트를 하나만 두고 이 과정 전체를 그 하나가 판단하게 만들 수도 있고, 검색 전담, 평가 전담, 답변 작성 전담처럼 역할을 나눈 여러 에이전트가 서로 결과를 주고받게 만들 수도 있는데, 앞의 구조를 단일 에이전트, 뒤의 구조를 다중 에이전트 Agentic RAG라 부른다[6].

이 방식의 핵심은 질문의 성격에 맞는 도구를 선택하는 것과, 근거가 부족하면 그대로 넘어가지 않고 재검색하는 것 두 가지로 요약된다. 정량 질문은 정확한 계산이 가능한 도구로 보내고 의미 질문은 검색으로 라우팅하며, 검색 결과가 빈약할 때 얼버무리지 않고 재시도하기 때문에 Naive RAG보다 신뢰도 높은 답을 생성한다.

4. Graph RAG(local/global)

Graph RAG는 문서들 사이의 관계를 지식그래프로 미리 구축해두는 방식이며, 지식그래프는 엔티티를 노드로, 관계를 엣지로 표현한 구조다. 이 그래프를 순회하거나 요약해 답을 생성하는데, 벡터 검색이 의미가 비슷한 것을 찾는 데 강점이 있다면 Graph RAG는 이것과 저것이 어떻게 연결되어 있는지를 답하는 데 강점이 있다.

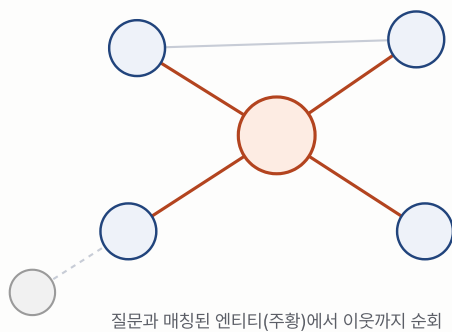
이 그래프는 질문이 들어오기 전에 미리 만들어둔다. 원본 문서에서 엔티티와 관계를 추출해서 그래프를 세우고, 그래프 전체를 서로 밀접하게 연결된 노드끼리 묶은 커뮤니티(community)로 계층적으로 나눈 뒤, 각 커뮤니티가 무엇을 담고 있는지 언어모델이 요약한 커뮤니티 리포트까지 미리 생성해둔다[7]. 질문이 들어오면 이렇게 미리 준비해둔 그래프와 리포트를 검색 방식에 따라 다르게 활용한다.

Graph RAG는 검색 방식에 따라 크게 두 가지로 나뉜다.

- Local search(지역 탐색)는 특정 엔티티 하나를 중심으로 그 이웃 노드들로 뻗어나가며(fan-out) 순회하는 방식으로[8], "React를 요구하는 공고와 함께 요구되는 기술은 무엇인가", "이 회사의 소재 지역은 어디인가"처럼 특정 대상에서 출발해 가까운 관계를 정확히 짚는 질문에 강하다.
- Global search(전역 탐색)는 그래프 전체를 앞서 만들어둔 커뮤니티 단위로 쪼갠 뒤, 각 커뮤니티 리포트를 맵리듀스 방식으로 훑어 질문과 관련도가 높은 리포트를 골라 종합해 답하는 방식이다[7][9]. "최근 프론트엔드 채용 시장의 전반적인 분위기는 어떠한가"처럼 개별 사실 하나가 아니라 그래프 전체를 조망해야 답할 수 있는 넓은 질문에 강하다.

정리하면 Local search는 나무 하나를 자세히 살펴보는 방식이고, Global search는 숲 전체를 구역별로 요약해 훑어보는 방식이다. 아래 그림은 이 두 탐색 방식의 차이를 그래프 구조로 보여준다.

Local search



Global search

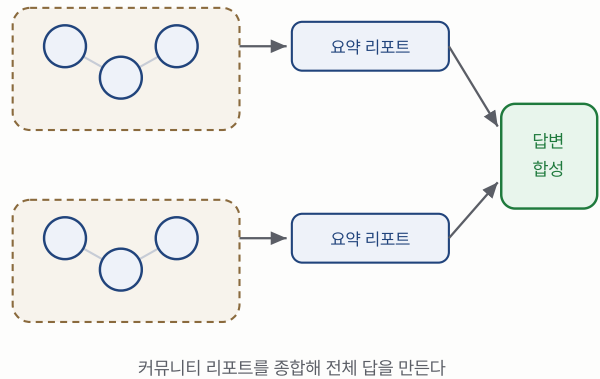


그림 2. Local search는 질문과 매칭된 엔티티(주황 원)에서 이웃 노드로 뻗어나가며 순회한다. 점선으로 이어진 먼 노드는 순회 범위 밖이다. Global search는 그래프를 커뮤니티(점선 사각형)로 나눠 미리 요약해둔 리포트를 모아 종합한다.

Graph RAG는 이 그래프와 커뮤니티 리포트를 미리 구축해야 하므로 색인 단계의 비용과 시간이 Naive RAG나 Agentic RAG보다 훨씬 크고, 원본 데이터가 바뀔 때마다 그래프를 다시 세우거나 갱신해야 한다는 부담이 있다. 다만 그 대가로 엔티티 사이의 관계를 정확하게 짚어내고, 그래프 전체를 조망하는 질문에도 답할 수 있다는 점에서 벡터 검색만으로는 다루기 어려운 영역을 커버한다.

5. 세 가지 비교

구분	Naive RAG	Agentic RAG	Graph RAG
검색 방식	벡터 검색 1회	라우터가 도구 선택 + 평가 후 재검색	그래프 순회(local) / 커뮤니티 요약(global)
강점	구현이 단순, 빠름	질문 성격에 맞는 도구 선택, 근거 부족 시 재시도	엔티티 간 관계, 구조적 질문에 정확
약점	정량, 관계 질문에 취약, 검증 없음	그래프형 관계 질문은 여전히 못 다룸	그래프 구축, 유지 비용 부담, 단독으로는 정량 계산 불가
적합한 질문 예	"이런 내용의 문서를 찾아달라"	"지금 상황을 종합해서 정리해달라"(도구 여러 개 필요)	"A와 함께 자주 나오는 것은 무엇인가", "최근 이 분야 전반
정직성 리스크	근거가 부족해도 답을 지어낼 수 있음	평가자가 부족함을 감지해 재검색/모름 처리 가능	관계는 정확하나 정량 집계는 별도 보강 필요

6. 상황별 선택 기준

세 방식은 우열 관계가 아니라 트레이드오프 관계이며, 이 트레이드오프를 정리하면 다음과 같다[2].

- Naive RAG는 답이 문서 안에 있고 속도가 중요할 때 적합하다. 빠르고 저렴하지만, 잘못된 청크를 가져와도 바로잡을 장치가 없다는 것이 근본 약점이다.
- Graph RAG는 법률, 컴플라이언스, 바이오처럼 지식이 구조적으로 얹혀 있을 때 적합하다. 구축과 갱신 비용이 크고 느리지만, 엔티티 사이의 관계를 정확히 다룬다.
- Agentic RAG는 다단계 추론과 자기교정이 필요할 때 적합하다. 가장 유연하고 강력하지만, 느리고 비싸며 디버깅이 어렵다.

여기서 얻는 교훈은 두 가지다. 첫째, 방식 선택은 질문의 성격이 결정한다는 것이고, 둘째, 강력한 방식일수록 대가가 따른다는 것이다. 비용, 지연, 디버깅 난이도가 함께 늘어나므로 필요 이상으로 무거운 방식을 남용하면 안 된다. 이 트레이드오프를 우리 서비스의 질문 분포에 대입한 결과가 다음 문서에서 다룰 하이브리드 설계다.

7. 우리의 선택

이 시스템은 세 가지 중 하나만 선택하지 않고, Agentic RAG의 도구 선택과 검증 구조 위에 도구 하나로 Graph RAG(local/global)를 결합한 하이브리드를 채택했다. 그 이유와 구체적인 구조는 다음 문서인 [04-our-architecture.md](#)에서 설명한다.

8. 참고 자료

1. [Retrieval-Augmented Generation for Large Language Models: A Survey](#)
2. [ByteByteGo, EP220: RAG vs Graph RAG vs Agentic RAG](#)
3. [Naive RAG: Why It Fails and How to Fix It](#)
4. [What is Agentic RAG?](#)
5. [Self-Reflective RAG with LangGraph](#)
6. [Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG](#)
7. [GraphRAG: Query-time overview](#)
8. [From Local to Global: A Graph RAG Approach to Query-Focused Summarization](#)
9. [GraphRAG: Improving global search via dynamic community selection](#)

우리 아키텍처: 하이브리드 Agentic + Graph RAG

1. 개요

앞선 세 문서에서 RAG의 기본 개념과 종류를 살펴봤고, 이 문서에서는 실제 구현 내용을 다룬다. 이 시스템은 나이브 RAG가 아니라 Agentic RAG와 Graph RAG를 결합한 **하이브리드**이며, 이 문서는 이 조합을 선택한 이유와 지식그래프의 절충 방식, 그리고 정직성을 코드 레벨에서 강제하는 방법을 차례로 다룬 뒤 마지막으로 전체 파이프라인을 순서대로 정리한다. 이 문서는 공고, 엔티티 테이블, 동시 요구 같은 데이터 용어를 자주 쓰는데, 그 정의는 [00-orientation.md](#)에 있다.

2. 하이브리드(Agentic+Graph) 선택 이유

이 서비스가 다루는 질문은 크게 세 종류로 구분된다. 첫째는 정량 질문으로, "채용 공고가 몇 건인가", "지역별 분포는 어떠한가"처럼 정확한 숫자가 필요한 경우다. 둘째는 의미 질문으로, "이 이력서와 비슷한 공고를 찾아달라"처럼 뜻이 비슷한 것을 찾아야 하는 경우다. 셋째는 관계 질문으로, "React와 함께 요구되는 기술은 무엇인가", "최근 프론트 시장 분위기는 어떠한가"처럼 엔티티 사이의 연결 구조를 파악해야 하는 경우다.

Naive RAG(벡터 검색 단일 방식)로는 이 세 종류를 모두 감당할 수 없다. 정량 질문에 벡터 검색을 쓰면 숫자가 부정확해질 수 있고, 관계 질문에 벡터 검색을 쓰면 "비슷한 문서"는 찾아도 "왜, 어떻게 연결되는지"는 답하지 못하기 때문이다. 이에 따라 다음 두 가지를 결합했다. 먼저 Agentic 구조를 채택해 질문마다 적절한 도구(SQL, 벡터, 그래프)를 라우터가 선택하게 했고, 근거가 부족하면 평가자가 재검색을 지시하도록 했다. 그리고 그 도구 중 하나로 Graph RAG를 포함시켜, 관계와 클러스터 질문에 정확히 답할 수 있도록 했다.

이는 "AI API만 떼다 쓴 게 아니다"라고 말할 수 있는 근거이기도 하다. 질문 하나를 처리하는 과정에서 라우팅 판단, 여러 도구 실행, 근거 충분성 검증, 필요 시 재검색까지 거치는데, 이런 구조는 단순 API 호출로는 나올 수 없다.

ByteByteGo가 정리한 "각 방식이 언제 적합한가"를 우리 질문 분포에 대입하면, 하이브리드가 필연적임이 분명해진다. 우리 서비스에는 세 성격의 질문이 모두 실제로 존재하기 때문이다.

질문 유형	예시	ByteByteGo가 권하는 방식	우리 구현
정량, 랭킹	"공고 몇 건인가", "상위 기술 1위는"	답이 데이터에 있고 속도 중요 → 결정론적 조회	sql_tool (Agentic이 라우팅)
의미 유사	"이 이력서와 비슷한 공고"	문서 안에 답, 속도 중요 → Naive RAG	vector_tool (BGE-M3 + pgvector)
관계, 구조	"React와 함께 요구되는 기술", "프론트 시장 분위기"	구조적 지식 → Graph RAG	graph_tool (local/global)
다단계, 자기교정	"지금 상황 종합 정리"	다단계 추론, 자기교정 → Agentic RAG	라우터+평가자 재검색 루프

한 방식만으로는 이 네 행을 모두 만족시킬 수 없었으므로, Agentic의 자기교정 골격을 바깥에 두고 그 도구 중 하나로 Graph를 넣는 조합을 택했다. 여기에는 강력한 방식일수록 대가가 크다는 원칙, 즉 노력과 비용과 디버깅 난이도가 커진다는 점도 함께 반영했다. 그래서 무거운 Agentic 루프를 매 질문에 무조건 돌리지는 않으며, 라우터가 정량 질문을 곧장 [sql_tool](#)로 보내 값싸고 정확하게 끝내도록 한 것이 그 예다.

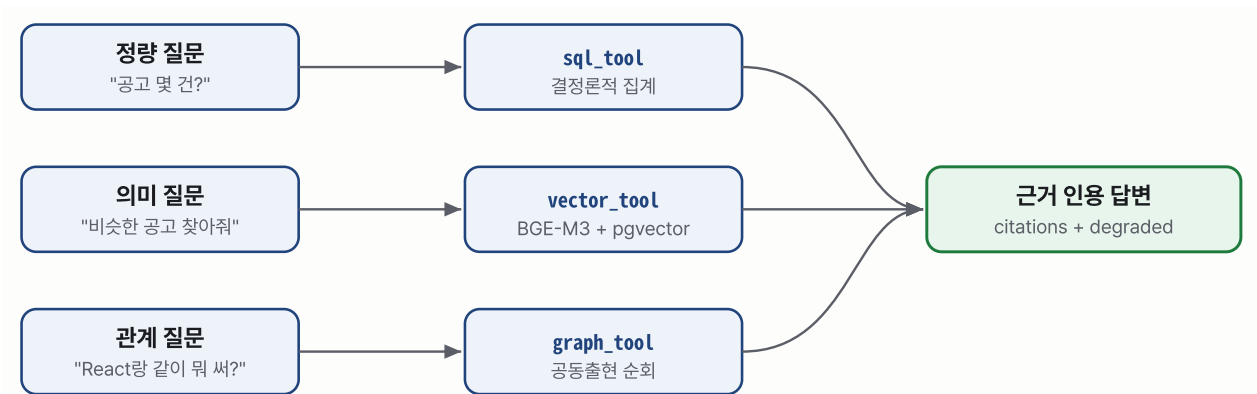


그림 3. 질문 유형마다 정해진 도구 하나로 고정 라우팅한 뒤, 세 경로 모두 같은 근거 인용·degraded 규약을 지키는 답변으로 합류한다. `router.py`의 `INTENT\_TOOLS` 매핑을 그대로 도식화한 것이다.

3. 지식그래프 A+ 절충

지식그래프 구축에는 두 가지 선택지가 있었고, 각각을 **방식 A**와 **방식 B**로 부른다.

- **방식 A(구조화 속성 그래프)**: 이미 보유한 구조화 데이터, 즉 공고, 기술, 자격증, 회사, 지역, 산업 테이블로 그래프를 직접 구축한다. 숫자와 관계가 전부 데이터에서 결정론적으로 나오므로 빠르고 정확하지만, "그래서 이 관계가 왜 중요한지"를 설명하는 서술이 없다.
- **방식 B(LLM 추출 그래프)**: 모든 것을 LLM에 맡겨 원본 텍스트에서 엔티티와 관계를 추출한다. 유연하지만 느리고 비용이 크며, LLM이 관계를 잘못 추출할 위험도 있다.

이 시스템은 방식 A를 뼈대로 그대로 채택하되, 방식 A의 유일한 약점인 "서술 없음"만 LLM으로 보강했다. 방식 A에 서술력을 한 단계 더한 조합이라는 뜻에서 이를 "**A+**" 방식이라 부른다. 학점 표기를 빌린 이름 그대로, "방식 A + 서술 보강"이 이 명칭의 전부다.

뼈대는 구조화 속성 그래프(방식 A)로, 서술만 LLM으로 보강한다. 숫자와 관계는 데이터에서 결정론적으로 추출하고, "그 관계가 어떤 의미인지"를 설명하는 문장만 LLM이 덧붙인다.

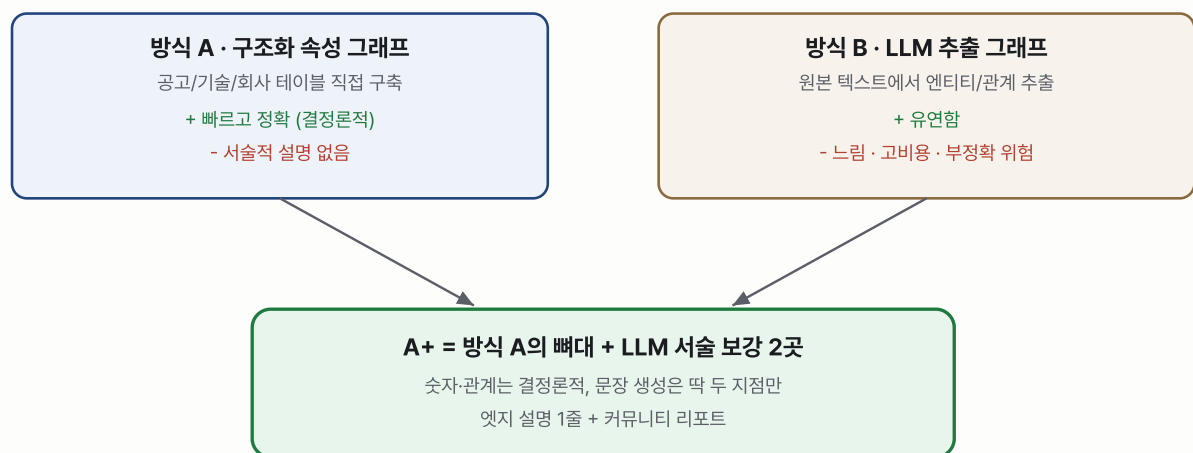


그림 4. 방식 A(구조화 속성 그래프)를 뼈대로 삼고, 방식 B(LLM 추출)의 장점인 서술력만 두 지점에 한정해 빌려온 것이 A+ 방식이다.

노드와 엣지는 다음과 같이 구성된다.

노드: Tech, Cert, Company, Region, Industry, Posting
엣지: Tech —CO_OCCURS(strength, n)— Tech (기술 동시 요구, 기존 co-occurrence 데이터 기반)
Tech —LEADS(lag, corr)— Tech (트렌드 선행 관계)
Posting —REQUIRES— Tech / Cert
Posting —POSTED_BY— Company —LOCATED_IN— Region —IN_INDUSTRY— Industry

저장은 Postgres 테이블(`graph_node`, `graph_edge`)에 하며, 순회는 재귀 CTE(SQL로 그래프를 따라가는 쿼리) 또는 오프라인에서 networkx/igraph 같은 그래프 라이브러리로 처리한다.

검색은 앞 문서에서 설명한 두 방식을 그대로 사용한다. Local search는 특정 기술의 이웃, 즉 함께 요구되는 기술과 요구되는 자격증, 주요 채용 회사를 순회해 정확한 수치로 답하는 방식으로, "React를 배우면 무엇을 함께 배워야 하는가" 같은 질문에 쓴다. Global search는 그래프를 커뮤니티로 분할해 답하는 방식인데, 커뮤니티는 Louvain이나 Leiden 알고리즘으로 검출하며 연산 비용이 커 Celery로 오프라인 처리한다. 각 커뮤니티는 LLM이 요약한 리포트로 생성되고, 넓은 질문에는 이 리포트들을 map-reduce 방식으로 종합해 답하는데, "최근 프론트 시장은 어떠한가" 같은 질문이 여기에 해당한다.

LLM을 사용하는 지점은 두 곳으로 제한했으며, 이것이 이 방식을 "A+"라 부르는 이유다. 하나는 엣지 설명 한 줄을 생성하는 것으로, 예를 들어 "React ↔ TypeScript 82% 동시 요구, 이 조합이 흔한 이유"를 설명하는 문장이다. 다른 하나는 커뮤니티 리포트, 즉 클러스터별 제목과 요약을 생성하는 것이다.

여기에 다음 원칙을 지킨다.

원본 공고 텍스트를 통째로 임베딩하지 않는다. 집계, 요약, 리포트 단위만 임베딩하여, 검색 결과가 항상 "정제된 근거"가 되도록 한다.

4. 정직성 가드

이 시스템 전체를 관통하는 불변 원칙이 있으며, 다음 네 가지는 코드 레벨에서 강제한다. 정량, 집계, 랭킹 질문은 무조건 SQL로 라우팅해 벡터나 그래프가 숫자를 지어내는 것을 원천 차단한다. 즉 "몇 건인가", "1위는 무엇인가" 같은 질문은 결정론적 SQL 집계 쿼리로만 답한다. 또한 답변마다 근거 인용(citations)을 강제하며, 검색이 빈약하면 "데이터 부족"으로 답하게 해 모른다고 정직하게 답하는 것도 정답 처리에 포함시킨다. LLM 폴백이 발생하면 `degraded: true` 로 정직하게 표기해, 정상 파이프라인이 아니라 대체 경로로 답했다는 사실을 숨기지 않는다. 마지막으로 DB에 없는 값, 예를 들어 합격 확률 같은 근거 없는 추정치는 절대 생성하지 않는다.

이 원칙은 `01-rag-basics.md` 에서 다룬 "근거 안에서만 답한다"는 원칙을 실제 구현으로 옮긴 것이며, 부가기능이라는 이유로 느슨하게 만들지 않고 본 서비스와 동일한 수준의 정직성을 요구한다.

5. 파이프라인 개요

전체 흐름은 다음과 같이 이어진다.

사용자 질문

|

▼

[1] Router/Planner (Claude Haiku)

| 질문 분해 → plan(intent, subqueries[], tools:[sql|vector|graph], pool)

▼

[2] Tools 실행 (병렬 가능)

├─ sql_tool 정량, 랭킹 결정론적 집계 쿼리 (100% 정확)
├─ vector_tool 의미 유사 BGE-M3 임베딩 + pgvector HNSW 코사인 + bge-reranker
└─ graph_tool 관계, 클러스터 지식그래프 local/global search

▼

[3] Evaluator (Claude) 근거 충분? pass / re-retrieve (도구, 파라미터 변경, 최대 2회)

▼

[4] Synthesis (Claude Sonnet) 근거 인용 강제 + JSON 구조화 출력

▼

프론트: steps[] 단계별 렌더 + tool_results[] 차트 파싱 + citations resolve

각 단계는 다음과 같이 요약된다. Router/Planner인 Claude Haiku는 질문을 읽고 필요한 도구를 계획하며, 정량 질문이면 `sql` 을 무조건 계획에 포함시킨다. Tools 단계에서는 계획에 따라 SQL, 벡터, 그래프 도구를 실행하는데, 필요하면 여러 도구를 동시에 실행할 수 있다. Evaluator인 Claude는 확보된 근거가 질문에 답하기 충분한지 점수를 매기고, 부족하면 도구나 파라미터를 바꿔 최대 2회까지 재검색한다. Synthesis인 Claude Sonnet은 근거를 인용하며 최종 답을 구조화된 JSON으로 생성하고, 근거가 끝내 부족하면 "모른다"고 답한다.

생성, 즉 자연어 답변 작성은 Claude API에 맡기는데 라우팅은 Haiku, 합성은 Sonnet이 담당한다. 반면 임베딩(BGE-M3)과 리랭킹(bge-reranker)은 로컬 GPU(RTX 4060 8GB)에서 직접 처리하며, 이처럼 로컬 자체 호스팅 파트와 API 파트를 명확히 구분했다. 이 구성 역시 "API만 떼다 쓴 게 아니다"를 뒷받침하는 근거 중 하나다.

프론트엔드는 이 파이프라인이 생성한 응답(`answer`, `route`, `plan`, `steps[]`, `tool_results[]`, `citations`, `confidence`, `degraded`)을 그대로 받아, 라우팅 판단부터 도구 실행, 검증, 합성까지의 과정을 단계별로 펼쳐 보여준다. "기계가 일하는 과정 자체"가 신뢰의 근거가 되도록 설계했다.

6. 시나리오로 보는 라우팅

개념 설명만으로는 각 도구가 실제로 언제, 어떻게 갈리는지 감이 잘 안 온다. 아래 네 가지는 실제 질문이 파이프라인을 통과하는 과정을 4단계(라우팅 → 도구 → 검증 → 합성) 순서 그대로 따라간 시나리오이며, 앞의 세 개는 정량·관계·의미 질문이 각각 다른 도구로 갈리는 정상 경로를, 마지막 하나는 근거를 못 찾았을 때 정직성 가드가 작동하는 경로를 보여준다. 프로세 설명과 구분되도록 코드블럭(트레이스 로그 형태)으로 표기한다.

시나리오 1 — 정량 질문: `sql_tool` 로 라우팅

질문: "React를 요구하는 공고가 몇 건이야?"

```
[1] Router   intent=skill_demand → tools=[sql]
[2] Tool     sql_tool.skill_demand(skill="React")
           → COUNT(DISTINCT posting_id) = 812건
[3] Eval     total_n=812 > 0 → pass · "근거 표본 812건"
[4] Synth    "채용 공고를 분석한 결과, React를 요구하는 공고는 **812건**이에요."
```

route: sql · confidence: high · degraded: false

시나리오 2 — 관계 질문: `graph_tool` 로 라우팅

질문: "React랑 같이 많이 쓰는 기술이 뭐야?"

```
[1] Router  intent=cooccurrence → tools=[graph]
[2] Tool    graph_tool.co_occurring_skills(skill="React")
           → 1-hop: TypeScript(82%) · Next.js(61%) · Redux(38%) ...
[3] Eval    items 존재 → pass · "근거 표본 4건"
[4] Synth   "React와 함께 가장 많이 요구되는 기술은 **TypeScript(82%)*,
           Next.js(61%), Redux(38%) 순이에요."
```

route: graph · confidence: high · degraded: false

시나리오 3 — 의미 질문: `vector_tool` 로 라우팅

질문: "백엔드 신입이 지원할 만한 공고 추천해줘"

```
[1] Router  intent=semantic_search → tools=[vector]
[2] Tool    vector_tool.semantic_search(query="...")
           → embed_query() (BGE-M3) → pgvector 코사인 top-8
           (is_tech_posting=true 필터로 비개발 공고 배제)
[3] Eval    items 8건 → pass · "근거 표본 8건"
[4] Synth   "조건에 맞는 공고를 8건 찾았어요: ..."
```

route: vector · confidence: medium · degraded: false

시나리오 4 — 근거 없음: 정직성 가드가 작동

질문: "COBOL을 요구하는 공고가 몇 건이야?"

```
[1] Router  intent=skill_demand → tools=[sql]
[2] Tool    sql_tool.skill_demand(skill="COBOL")
           → resolve_skill() 실패 (skill 테이블에 없는 이름) → None
           → tool_outputs = []
[3] Eval    tool_outputs 비어있음 → fail
           "근거 없음 — 도구가 결과를 반환하지 않음"
[4] Synth   passed=False → "관련 데이터가 부족해요."
```

route: sql · confidence: 0 · degraded: true

네 시나리오를 나란히 보면 이 구조의 핵심이 드러난다. 앞의 세 시나리오는 질문 성격에 따라 도구만 바뀔 뿐 라우팅 → 도구 → 검증 → 합성이라는 흐름 자체는 동일하고, 마지막 시나리오는 그 흐름 중 어느 단계에서든 근거가 없으면 답을 지어내지 않고 `degraded: true` 와 함께 "부족하다"고 정직하게 멈춘다. `evaluate()` 가 빈 `tool_outputs` 를 즉시 실패로 판정하는 코드가 실제로 이 네 번째 경로를 만든다.

7. 실제 코드로 확인하는 RAG 유형

지금까지는 설계 관점에서 설명했다. 여기서는 실제로 배포된 소스 코드를 근거로, 이 시스템이 `03-rag-types.md` 에서 정리한 분류 중 정확히 어디에 해당하는지 확인한다. 아래 코드는 전부 `backend/app/services/rag/` 아래 실제 파일에서 그대로 발췌했다.

7.1 라우팅: 질문 성격에 따라 도구를 고른다

`backend/app/services/rag/router.py` 의 `INTENT_TOOLS` 는 질문의 의도(intent) 하나를 도구 하나에 매핑한다. 정량 질문은 `sql` 로, 관계 질문은 `graph` 로, 의미 질문은 `vector` 로 고정해서 보내는 코드가 그대로 있다.

```
# backend/app/services/rag/router.py
INTENT_TOOLS = {
    "cooccurrence": ["graph"],
    "skill_demand": ["sql"],
    "skill_ranking": ["sql"],
    "compare": ["sql"],
    "concept_ranking": ["sql"],
    "cert_ranking": ["sql"],
    "semantic_search": ["vector"],
    "overview": ["sql"],
    "region_distribution": ["sql"],
}

def plan(session: Session, llm: LLMClient, question: str, pool: str | None) -> tuple[Plan, bool]:
    """(Plan, degraded). LLM 성공 시 degraded=False, 폴백 시 True."""
    raw = llm.json(_PLANNER_SYSTEM, question, temperature=0.0)
    if not raw or raw.get("intent") not in INTENT_TOOLS:
        return _heuristic(session, question, pool), True
    ...
```

LLM이 의도 분류를 내면 그 결과로 도구를 정하고, LLM 호출이 실패하면 `_heuristic()` 이라는 키워드 기반 폴백으로 넘어가 `degraded=True` 를 반환한다. 이 "질문마다 도구를 스스로 고르고, 실패를 감추지 않고 표시한다"는 두 가지가 Agentic RAG를 Naive RAG와 가르는 핵심이며, 이 프로젝트에서는 라우터 계층에 그대로 코드화되어 있다.

7.2 도구: SQL, 벡터, 그래프 세 갈래

정량 질문은 `backend/app/services/rag/tools/sql_tool.py` 가 결정론적 SQL 집계로 처리한다.


```
# backend/app/services/rag/tools/sql_tool.py
def skill_demand(
    session: Session,
    skill_name: str,
    pool: str | None = None,
    category: str | None = None,
    entry_level: bool = False,
    verbose: bool = False,
) -> dict | None:
    resolved = resolve_skill(session, skill_name)
    if not resolved:
        return None
    skill_id, canonical = resolved
    ...
    sql = (
        f"SELECT COUNT(DISTINCT pt.posting_id) FROM posting_tech pt "
        f"JOIN posting p ON p.id = pt.posting_id "
        f"{join}"
        f"WHERE pt.skill_id = :sid AND pt.is_deleted = false AND {_POOL_WHERE}{where_extra}"
    )
```

의미 질문은 `backend/app/services/rag/tools/vector_tool.py` 가 BGE-M3 임베딩과 pgvector 코사인 거리로 처리한다.

```
# backend/app/services/rag/tools/vector_tool.py
def semantic_search(
    session: Session, query: str, pool: str | None = None, limit: int = 8, verbose: bool = False
) -> dict | None:
    vec = embed_query(query)
    if vec is None:
        return None

    qv = "[" + ", ".join(f"{x:.6f}" for x in vec) + "]"
    sql = (
        f"SELECT p.id, p.title, p.company, p.pool, "
        f"(e.embedding <=> CAST(:qv AS vector)) AS dist "
        f"FROM posting_embedding e "
        f"JOIN posting p ON p.id = e.id "
        f"WHERE {_POOL_WHERE} "
        f"ORDER BY e.embedding <=> CAST(:qv AS vector) LIMIT :limit"
    )
```

관계 질문은 `backend/app/services/rag/tools/graph_tool.py` 가 처리하는데, 여기서 앞서 설명한 설계와 실제 구현 사이에 짝여야 할 차이가 하나 있다. 이 함수의 docstring은 스스로를 이렇게 밝힌다.

```
# backend/app/services/rag/tools/graph_tool.py
"""graph_tool — 지식그래프 local search(공동출현 순회).

"React 배우면 뭘 같이?" 류 관계 질문에 정확한 수치로 답한다.
엣지 = 같은 공고에서 함께 요구된 기술 쌍. strength = 대상 기술 공고 중 동반 비율.
서브그래프(nodes/edges)를 tool_result.graph 로 반환해 프론트 네트워크 위젯이 렌더.
2-hop 크로스엣지: 1-hop 이웃들끼리의 공동출현도 함께 반환해 2단 네트워크를 구성한다.
"""

def co_occurring_skills(
    session: Session, skill_name: str, pool: str | None = None, limit: int = 8, verbose: bool = False
) -> dict | None:
    ...
    sql_1hop = (
        f"SELECT s2.canonical, s2.id, COUNT(DISTINCT pt2.posting_id) n "
        f"FROM posting_tech pt1 "
        f"JOIN posting_tech pt2 ON pt1.posting_id = pt2.posting_id "
        f" AND pt2.skill_id <> pt1.skill_id AND pt2.is_deleted = false "
        f"JOIN skill s2 ON s2.id = pt2.skill_id "
        ...
    )
```

`co_occurring_skills` 는 위 "지식그래프 A+ 절충"에서 그림으로 설명한 `graph_node` / `graph_edge` 같은 영속 그래프 테이블을 순회하지 않는다. `posting_tech` 와 `skill`, `posting` 세 관계형 테이블을 요청이 들어올 때마다 자기 조인(self-join)해서, "같은 공고에 함께 등장한 기술 쌍"이라는 서브그래프를 즉석에서 계산해 반환한다. 즉 Graph RAG의 `local search`가 노리는 결과(엔티티 하나에서 이웃으로 뻗어나가는 순회)는 그대로 내지만, 그 방법은 별도 그래프 저장소가 아니라 SQL 자기조인이다. 커뮤니티 리포트를 미리 만들어두고 맵리듀스로 종합하는 `global search`는 이 파일에 아직 구현되어 있지 않다.

7.3 평가와 합성: 근거 검증과 인용 강제

`backend/app/services/rag/evaluator.py` 는 파일 전체가 18줄이다. 도구가 실제로 근거를 냈지만 결정론적으로 판정한다.

```
# backend/app/services/rag/evaluator.py
"""Evaluator — 검색 근거 충분성 판정.

증분 1은 결정론적: 도구가 실제 근거(n>0, 항목 존재)를 냈으면 pass.
설계상 재검색 루프(최대 2회)는 후속 증분에서 LLM 평가로 확장한다.
"""

def evaluate(tool_outputs: list[dict]) -> tuple[bool, str]:
    if not tool_outputs:
        return False, "근거 없음 — 도구가 결과를 반환하지 않음"
    total_n = sum(o.get("n", 0) for o in tool_outputs)
    has_items = any(o.get("tool_result", {}).get("items") for o in tool_outputs)
    if total_n <= 0 and not has_items:
        return False, "근거 표본 0 — 데이터 부족"
    return True, f"pass · 근거 표본 {total_n:,}건"
```

`docstring`이 스스로 밝히듯, 지금 배포된 버전은 "도구가 낸 근거가 있는지 없는지"만 판정하는 1회성 결정론적 게이트다. 위에서 설명한 "부족하면 도구나 검색어를 바꿔 재검색하는" 자기교정 루프(최대 2회)는 아직 이 코드에 없고, 후속 증분 과제로 남아 있다.

근거가 통과되면 `backend/app/services/rag/synthesis.py` 가 답을 생성하는데, 여기서는 근거 인용을 강제하는 프롬프트와, LLM이 근거를 무시하고 "데이터 부족"이라고 얼버무릴 때 이를 사실 템플릿으로 되돌리는 방어 코드를 확인할 수 있다.

```
# backend/app/services/rag/synthesis.py
_SYNTH_SYSTEM = (
    "너는 채용시장 데이터 어시스턴트다. 아래에 주어진 '사실'만 근거로 한국어 2~3문장 답을 작성한다. "
    "사실에 없는 수치나 항목을 절대 지어내지 마라. "
    ...
)

def synthesize(
    llm: LLMClient, question: str, tool_outputs: list[dict], passed: bool
) -> tuple[str, bool, bool]:
    facts = [o["facts"] for o in tool_outputs if o.get("facts")]
    if not passed or not facts:
        return "관련 데이터가 부족해요.", True, False
    ...
    text = llm.text(_SYNTH_SYSTEM, prompt, temperature=0.3)
    if text and text.strip() and not _is_bail(text.strip()):
        return text.strip(), False, True
    # LLM이 미가용이거나, 사실이 있는데도 부족 문구로 답했다면 사실 템플릿으로 덮어서
    # 실제 데이터를 보여준다(허위 '부족' 응답 방지).
    return _fallback(facts), True, True
```

`_is_bail()` 이 하는 일이 특히 이 시스템의 정직성 원칙을 코드로 보여준다. LLM이 근거를 받고도 습관적으로 "데이터가 부족하다"는 문구를 내면, 그 답을 그대로 쓰지 않고 실제 근거 문장으로 덮어서 버린다. 근거가 있는데 없다고 둘러대는 것도, 근거가 없는데 있다고 지어내는 것도 둘 다 막는 구조다.

7.4 오케스트레이션: 라우팅 → 도구 → 평가 → 합성

이 네 단계를 순서대로 묶는 곳은 `backend/app/services/rag/pipeline.py` 의 `run_chat_events()` 다.

```
# backend/app/services/rag/pipeline.py
def run_chat_events(
    session: Session, question: str, pool: str | None = None, *,
    verbose: bool = False, collect: dict[str, Any] | None = None,
) -> Iterator[dict[str, Any]]:
    ...
    llm = get_llm()
    p, plan_degraded = make_plan(session, llm, question, pool)      # [1] 라우팅
    ...
    tool_outputs, fell_back = _dispatch(session, p, verbose=verbose) # [2] 도구 실행
    ...
    passed, eval_detail = evaluate(tool_outputs)                    # [3] 근거 검증
    ...
    answer, synth_degraded, answered = synthesize(llm, question, tool_outputs, passed) # [4] 합성
```

이 함수를 그대로 노출하는 엔드포인트가 `backend/app/routers/chat.py` 다.

```
# backend/app/routers/chat.py
"""POST /chat — 하이브리드 Agentic + Graph RAG 엔드포인트(v2 구조화 JSON)."""

@router.post("/chat", response_model=ChatResponse)
def chat(body: ChatRequest, session: SessionDep) -> ChatResponse:
    return run_chat(session, body.question, body.pool, verbose=body.verbose)

@router.post("/chat/stream")
def chat_stream(body: ChatRequest, session: SessionDep) -> StreamingResponse:
    def gen() -> Iterator[str]:
        for event in run_chat_events(session, body.question, body.pool, verbose=body.verbose):
            yield f"data: {json.dumps(event, ensure_ascii=False)}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")
```

7.5 종합: 어떤 RAG인가, 그리고 지금 어디까지 왔는가

코드를 근거로 정리하면, 이 시스템은 라우팅과 도구 선택, 근거 검증, 합성이라는 Agentic RAG의 뼈대를 갖췄고, 그 도구 중 하나 ([graph_tool](#))가 Graph RAG의 local search 아이디어를 관계형 데이터 자기조인으로 구현한 하이브리드다. [03-rag-types.md](#)의 분류에 정확히 대입하면 "Agentic RAG 골격 위에 Graph RAG 스타일 도구 하나를 얹은 구조"에 해당한다.

다만 이 문서 앞부분에서 설명한 완성형 설계와 지금 배포된 코드 사이에는 정직하게 밝혀야 할 차이가 있다. [backend/app/services/rag/llm.py](#)의 실제 LLM 백엔드는 Gemini이며([GeminiClient](#)), "설계상 나중에 Claude로 교체 가능하도록 인터페이스를 좁게 둔다"는 주식대로 아직은 추상화 인터페이스 뒤에 Gemini가 배선된 상태다. 지식그래프도 앞서 설명한 [graph_node / graph_edge](#) 영속 테이블이 아니라, 매 요청마다 관계형 테이블을 자기조인해 즉석으로 서브그래프를 계산하는 방식이며, 커뮤니티 리포트 기반 global search는 아직 구현되지 않았다. 근거 검증(validator)도 LLM이 판단해 재검색을 지시하는 루프가 아니라, 근거 유무만 보는 결정론적 1회 게이트다. 이 격차를 감추지 않고 그대로 적어두는 것 자체가, 위 "정직성 가드" 절에서 강제한 원칙(모르면 모른다고 답하고, degraded 상태를 숨기지 않는다)을 이 문서 자신에도 똑같이 적용한 것이다.

RAG의 최전선. 검색을 넘어, LLM이 지식을 직접 관리한다

1. 개요

앞선 문서들이 다룬 RAG는 모두 질문이 들어오면 그때 문서를 검색한다는(retrieval at query time) 공통 전제 위에 서 있었다. 이 문서에서는 그 전제를 뒤집는 최근의 접근을 다루는데, LLM이 지식베이스를 미리 직접 작성하고 유지하면서 질문에는 그렇게 정제된 위키를 참조해 답하는 방식이다. Andrej Karpathy가 제시한 "LLM-Wiki" 패턴이 이를 대표하며, 흥미롭게도 이 패턴은 우리 [rag-development](#) 문서 체계 자체가 답아 있는 구조이기도 하다. 그래서 이 문서에서는 개념을 정리한 뒤 우리 작업과의 관계를 짚어보기로 한다. 공고, 집계, 리포트 같은 용어의 바탕은 [00-orientation.md](#) 와 [04-our-architecture.md](#) 에 있다.

2. 검색형 RAG의 한계

검색형 RAG는 매 질문마다 원문 청크를 새로 꺼내 오는 구조라서, 여기에는 몇 가지 구조적 비효율이 따라붙는다. 우선 같은 주제를 열 번 물으면 열 번 모두 원문에서 관련 청크를 긁어 족석에서 종합하게 되므로, 지식이 쌓이지 않고 매번 처음부터 다시 종합하는 셈이 된다. 게다가 서로 어긋나는 두 문서가 있어도 검색은 그저 유사한 청크를 반환할 뿐이어서, 이 둘이 충돌한다는 사실 자체를 알려주지 못한다. 문서와 문서 사이의 연관, 예컨대 이 개념이 저 사건의 원인이라는 관계 같은 것도 청크 단위 검색에서는 잘 드러나지 않아 그대로 사라지고 만다.

3. LLM-Wiki 패턴

Karpathy가 제안하는 것은 검색이 아니라 **점진적 종합**이다(incremental synthesis). 새 자료가 들어오면 LLM이 그것을 읽고 요점을 추출해 기존 위키 페이지에 통합하며, 그 과정에서 교차 참조를 갱신하고 모순이 있으면 표시해 둔다. 그 결과 위키는 자료가 쌓일수록 오히려 더 풍부해지는, 이른바 "살아 있는 종합"이 되어간다. 이 패턴의 핵심 통찰은 다음 한 문장으로 요약된다.

지식베이스를 유지하는 일에서 정작 고된 부분은 읽거나 사고가 아니라 **북키퍼링**이다(bookkeeping). LLM은 지치지 않고 교차 참조와 일관성 관리를 대신 수행해 주므로, 사람들이 포기하곤 했던 위키 유지의 비용이 거의 0에 가깝게 낮아진다.

이 패턴은 세 계층으로 구성된다.

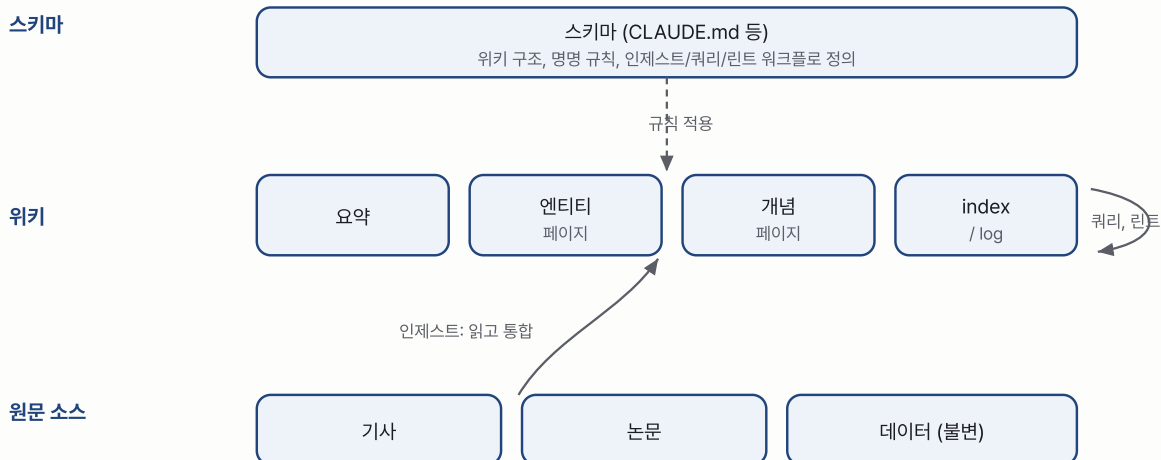


그림 1. LLM-Wiki 3계층. 불변의 **원문 소스**를 LLM이 인제스트해 **위키**(요약, 엔티티, 개념, index/log)로 종합하고, **스키마**가 그 구조와 규칙을 규정한다. 작도는 [Karpathy의 LLM-Wiki gist](#)를 참고했다.

세 계층은 각각 역할이 뚜렷이 나뉜다. **원문 소스**는 기사, 논문, 데이터 등 변경하지 않는 진실의 원천으로, 위키의 내용이 틀렸다고 판단될 때는 언제나 이곳으로 되돌아가 대조하게 된다. **위키**는 LLM이 생성하고 유지하는 마크다운 페이지들로 이루어지는데, 요약과 엔티티 페이지, 개념 페이지, 그리고 탐색을 돕는 [index.md](#) (내용 목록)와 [log.md](#) (인제스트, 쿼리, 린트의 시간순 기록)로 구성된다. **스키마**는 위키의 구조와 규칙, 작업 절차를 LLM에게 알려주는 설정 문서로, 위키가 일관된 형태를 유지하도록 기준을 제공한다.

주요 작업은 인제스트, 쿼리, 린트 세 가지로 나뉜다. 인제스트는 새 소스를 읽고 한 번에 10~15개 페이지를 갱신하는 작업이고, 쿼리는 관련 페이지를 찾아 인용과 함께 답한 뒤 그 과정에서 얻은 가치 있는 발견을 다시 위키에 반영하는 작업이며, 린트는 모순, 고아 페이지, 오래된 주장, 누락된 교차 참조를 점검하는 작업이다. 이 패턴은 Vannevar Bush가 1945년에 구상한 Memex를 정신적 선조로 삼는데, 문서 간의 연상적 연결을 갖춘 개인의 정제된 지식 저장소라는 발상이 그 뿌리에 있다.

4. 우리 작업과의 관계

이 패턴을 지금 당장 도입하려는 것은 아니지만, 두 가지 이유에서 기록해 둘 가치가 있다.

첫째, 우리 [rag-development](#) 문서 체계 자체가 이미 **LLM-Wiki의 축소판**에 해당한다. 채팅에서 결정한 내용을 개념서([rag-learning/](#))와 구현기([rag-build-journal/](#))로 종합하고, 뷰어의 목차가 [index](#) 역할을 하며, 자료가 들어올 때마다(이 문서가 바로 그 예다) 관련 페이지를 갱신해 나간다. 결국 북키핑을 LLM이 대신한다는 발상을 우리는 이미 실천하고 있는 셈이다.

둘째, 우리 RAG 아키텍처의 "A+" 원칙과도 철학이 통한다. 우리는 원본 공고 텍스트를 통째로 임베딩하지 않고 집계, 요약, 리포트 단위만 임베딩하기로 결정했는데([04-our-architecture.md](#)), 이는 검색 결과가 항상 정제된 근거가 되도록 한다는 뜻이다. LLM-Wiki가 원문 대신 정제된 위키를 참조하는 방향과 같은 지향이며, 커뮤니티 리포트를 미리 생성해 두는 Graph RAG의 global search 역시 질의 시점이 아니라 사전에 종합해 둔다는 점에서 이 패턴과 맞닿아 있다.

정리하면 LLM-Wiki는 검색이나 종합이냐라는 축에서 RAG의 다음 단계를 보여주는 참조점이고, 우리는 정제된 단위 임베딩, 사전 종합 리포트, 문서 자동 종합이라는 형태로 이미 그 지향의 일부를 채택하고 있다.

5. 참고 자료

각 자료가 무엇이고 현재 어떻게 다뤄지고 있는지를 함께 적는다.

자료	무엇인가	현재 소
ByteByteGo, EP220 "RAG vs Graph RAG vs Agentic RAG"	세 RAG 방식의 파이프라인, 강약점, 적합 상황을 비교한 인포그래픽	반영 온
Andrej Karpathy, LLM-Wiki gist	LLM이 위키를 직접 작성하고 유지하는 영속적 지식베이스 방법론(3계층, 인제스트/쿼리/린트)	반영 온
GeekNews(hada.io) topic 28208	위 Karpathy gist의 한국어 소개글	반영 온
YouTube @sv.developer (에이전틱 AI)	에이전틱 AI 개념을 다루는 채널	참조 프
YouTube @the.brain.trinity (LLM 위키: 옵시디언)	옵시디언 기반으로 LLM 위키를 구축하는 실천 채널	참조 프

유튜브 채널 두 곳은 특정 영상 URL이 아니라 채널 전체 링크이므로, 개별 주장으로 인용하기보다는 학습 경로의 방향 표지로 남겨 둔다. 나중에 특정 회차를 지정하게 되면 그 내용을 개념서와 구현기에 반영할 예정이다.

고급 검색: 임베딩 학습, 하이브리드, 리랭킹, 인덱스 튜닝

1. 개요

이 문서는 앞의 [02-embeddings-vectordb.md](#) 가 다룬 임베딩과 벡터 검색의 기본 위에, 실제 검색 품질을 끌어올릴 때 쓰는 네 가지 기법을 초보자 눈높이로 설명한다. 첫째로 임베딩 모델이 애초에 어떻게 학습되는지를 조금 더 깊이 보고, 둘째로 키워드 검색과 의미 검색을 합치는 하이브리드 검색을 다루며, 셋째로 검색 결과를 다시 정렬하는 리랭킹을 설명하고, 넷째로 벡터 인덱스인 HNSW의 파라미터를 어떻게 조율하는지 정리한다. 각 절은 개념을 먼저 직관으로 잡은 다음, 코드와 그림으로 구체화한다.

2. 1. 임베딩 학습 방식

02 문서에서 임베딩은 의미가 가까운 문장을 가까운 벡터로 만든다고 했다. 그런데 모델은 무엇이 가깝고 무엇이 먼지를 어떻게 배울까. 답은 대조학습이라 불리는 방식이다.

대조학습의 핵심은 당겨야 할 쌍과 밀어내야 할 쌍을 모델에게 보여주고, 당길 것은 가깝게 밀 것은 멀게 되도록 벡터를 조정하는 것이다[1]. 서로 의미가 통하는 두 문장을 양성 쌍이라 부르고, 관계가 없는 문장을 음성이라 부른다. 학습은 양성 쌍의 벡터 거리는 줄이고 음성과의 거리는 늘리는 방향으로 진행된다[2].

여기서 음성을 어떻게 구하느냐가 효율을 좌우한다. 가장 널리 쓰는 방법이 배치 내 음성이다. 한 번에 M개의 예시를 묶어 학습할 때, 각 예시의 진짜 짝만 양성으로 두고 같은 배치의 나머지 M에서 1을 뺀 예시를 전부 음성으로 재사용하는 것이다[2]. 음성을 따로 만들 필요 없이 배치 안에서 공짜로 얻으므로 계산이 매우 효율적이다. 여기에 더해, 겉보기에는 질문과 비슷하지만 실제로는 답이 아닌 까다로운 예시를 일부러 골라 넣기도 하는데 이를 하드 네거티브라 부른다. 쉬운 음성만으로는 모델이 대충 구분해도 통과하지만, 헛갈리는 음성을 섞으면 미세한 의미 차이까지 배우도록 강제된다[1].

대조학습: 양성은 당기고 음성은 민다



이 학습은 보통 두 단계로 이루어진다. 먼저 방대한 텍스트 쌍으로 대조 사전학습을 해서 넓은 의미 감각을 익히고, 그다음 좀 더 정제된 데이터로 대조 미세조정을 해서 검색 같은 특정 작업에 맞춘다[2]. 우리가 쓴 BGE-M3도 이런 대조학습으로 훈련된 다국어 임베딩 모델이라, 한국어와 영어가 섞인 공고에서도 의미가 통하는 것끼리 가깝게 놓는다. **02** 문서의 EV충전과 전기차충전이 0.84로 가까웠던 것이 바로 이 학습의 결과다.

3. 2. 하이브리드 검색: 키워드와 의미의 결합

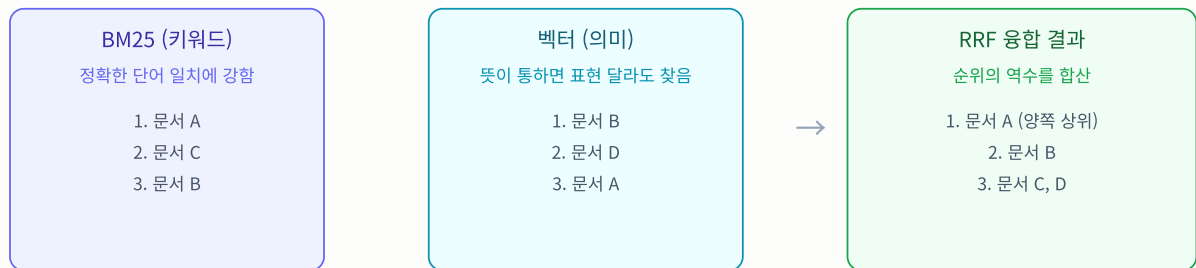
벡터 검색은 의미가 비슷한 것을 잘 찾지만 약점이 있다. 정확히 일치해야 하는 희귀 단어, 예를 들어 제품 코드나 고유한 기술명 같은 것은 의미가 아니라 글자 그대로 맞아야 하는데, 벡터는 이런 정확한 일치를 놓치기 쉽다[3]. 반대로 오래된 키워드 검색 방식인 BM25는 단어가 정확히 겹치는 문서를 잘 찾지만 의미는 전혀 이해하지 못한다. 뜻이 같아도 표현이 다르면 못 찾는다[3].

두 방식의 약점이 서로 반대라서, 둘을 합치면 서로의 사각지대를 메운다. 이것이 하이브리드 검색이다. 키워드 기반의 희소 검색인 BM25와 의미 기반의 밀집 벡터 검색을 각각 돌린 뒤, 두 결과를 하나의 순위로 합친다[4].

합칠 때 문제가 하나 있다. 두 점수의 척도가 다르다는 것이다. BM25 점수는 상한이 없는 양수인데 코사인 유사도는 -1에서 1 사이라, 그냥 더하거나 평균 내면 한쪽이 다른 쪽을 압도해버린다[3]. 그래서 점수 대신 순위만 쓰는 방법을 쓴다. 상호 순위 융합이라 불리는 RRF다. 각 문서에 대해 두 목록에서의 순위의 역수를 더해 최종 점수를 매기는 방식이라, 척도가 다른 문제를 애초에 피한다[3][4].

```
def rrf(list_bm25, list_vector, k=60):
    score = {}
    for rank, doc in enumerate(list_bm25):
        score[doc] = score.get(doc, 0) + 1 / (k + rank)
    for rank, doc in enumerate(list_vector):
        score[doc] = score.get(doc, 0) + 1 / (k + rank)
    return sorted(score, key=score.get, reverse=True)
```

하이브리드 검색: 두 목록을 순위로 융합



두 검색 모두에서 높은 문서가 최종 상위로 올라온다

우리 프로젝트는 현재 밀집 벡터만 쓰지만, BGE-M3는 원래 밀집과 희소를 함께 낼 수 있는 모델이라 하이브리드로 확장할 여지가 있다. 지금은 규칙 기반 SQL이 정확한 키워드 집계를 맡고 벡터가 의미 검색을 맡는 식으로 역할을 나눠, 하이브리드의 이점 일부를 다른 방식으로 얻고 있다.

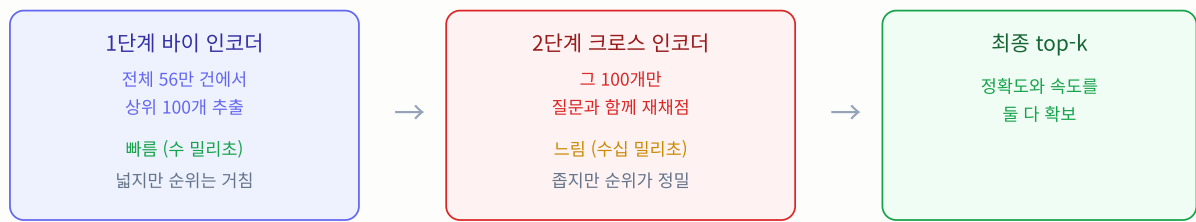
4. 3. 리랭킹: 정밀 재정렬

벡터 검색으로 뽑은 상위 문서가 항상 완벽한 순위는 아니다. 벡터 검색은 질문과 문서를 각각 따로 벡터로 만든 뒤 비교하는데, 이런 모델을 바이 인코더라 부른다. 문서 벡터를 미리 만들어두므로 수백만 건도 수 밀리초에 훑는 속도가 강점이지만, 질문과 문서를 따로 압축하다 보니 둘 사이의 미세한 관련성을 놓칠 수 있다[5][6].

이 순위를 다시 매기는 것이 리랭킹이고, 여기에 크로스 인코더를 쓴다. 크로스 인코더는 질문과 문서를 하나로 이어붙여 함께 모델에 넣어서, 질문의 모든 토큰이 문서의 모든 토큰을 살펴본 뒤 관련도 점수를 낸다[5][6]. 훨씬 정확하지만 쌍 하나마다 모델을 한 번씩 돌려야 해서 느리다[7].

그래서 실무는 두 단계로 나눈다. 먼저 빠른 바이 인코더로 전체에서 상위 후보 100개를 몇 밀리초에 추리고, 그다음 느리지만 정확한 크로스 인코더로 그 100개만 다시 채점해 순위를 바로잡는다[7]. 전수에 크로스 인코더를 쓰면 불가능하지만, 100개로 좁힌 뒤 쓰면 감당할 만하다.

2단계 검색: 넓게 추린 뒤 정밀하게 재정렬



리랭킹에 자주 쓰는 오픈 모델이 BGE 계열의 리랭커다. bge-reranker-v2-m3는 다국어를 지원하고 크기가 작아서 후보 100개 정도는 CPU에서도 돌릴 수 있다[8]. 우리 설계도 임베딩은 BGE-M3, 리랭킹은 같은 계열의 리랭커를 쓰는 조합을 염두에 두었다.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("BAAI/bge-reranker-v2-m3")

# 1단계에서 벡터로 추린 후보들
candidates = ["백엔드 개발자 공고 ...", "프론트엔드 공고 ...", "..."]
pairs = [(query, doc) for doc in candidates]
scores = reranker.predict(pairs) # 질문-문서 쌍마다 관련도 점수
ranked = [doc for _, doc in sorted(zip(scores, candidates), reverse=True)]
```

5. 4. HNSW 인덱스 파라미터 튜닝

02 문서에서 전수 비교는 느려서 근사 최근접 이웃을 쓰고, 그 대표가 HNSW라고 했다. HNSW는 계층 그래프를 미리 만들어 두고 그 위를 건너뛰며 탐색하는데, 그래프를 얼마나 촘촘하게 만들지와 탐색을 얼마나 넓게 할지를 파라미터로 조절한다. pgvector 기준으로 세 개가 핵심이다[9][10].

파라미터	언제 쓰나	뜻	기본값	올리면
m	인덱스 생성 시	노드당 연결 수	16	정확도 상승, 메모리와 빌드 시간 증가
ef_construction	인덱스 생성 시	빌드 때 후보 목록 크기	64	그래프 품질 상승, 빌드 시간 증가
ef_search	검색 시	질의 때 후보 목록 크기	40	재현율 상승, 검색 속도 저하

m은 그래프에서 각 노드가 몇 개의 이웃과 연결되는지를 정한다. 크게 잡으면 고차원 데이터에서 재현율이 오르지만, 연결이 많아지는 만큼 메모리를 더 쓴다. 노드 N개에 대해 연결 목록만으로도 대략 4 곱하기 m 곱하기 N 곱하기 1.1 바이트가 들어서, m을 키우면 대규모 데이터에서 메모리가 병목이 된다[9]. 그래서 보통 12에서 48 사이에서 시작하고, 대규모에서는 16에서 24 정도를 권장한다[9][10].

ef_construction은 인덱스를 만들 때 이웃 후보를 얼마나 넓게 살펴볼지를 정한다. 높이면 더 좋은 그래프가 나오지만 빌드가 느려지고, 어느 선을 넘으면 품질은 거의 안 오르고 시간만 늘어나므로 96에서 128 정도가 실용적이다[9][10].

ef_search는 실제 검색 시점의 조절 손잡이다. 이 값을 키우면 더 많은 후보를 살펴 정확한 결과를 얻지만 그만큼 느려지고, 줄이면 빨라지지만 관련 문서를 놓칠 수 있다. m과 ef_construction은 인덱스를 다시 만들어야 바뀌지만 ef_search는 질의마다 즉석에서 바꿀 수 있어서, 정확도와 속도의 균형을 잡는 1차 손잡이로 쓴다[10].

```
-- HNSW 인덱스 생성. 코사인 거리 기준.  
CREATE INDEX ON posting_embedding  
  USING hnsw (embedding vector_cosine_ops)  
  WITH (m = 16, ef_construction = 128);  
  
-- 이 세션의 검색 정확도-속도 균형을 조절.  
SET hnsw.ef_search = 100;
```

우리 프로젝트는 아직 벡터 검색을 기능 플래그로만 열어 둔 상태라 HNSW 인덱스를 프로덕션에 올리진 않았고, 따라서 우리 데이터에서의 빌드 시간과 질의 지연 실측은 아직 없다. 이 수치는 벡터 검색을 실제로 켜는 시점에 측정해 [00-environment.md](#)의 해당 항목과 함께 채운다. 다만 위의 권장 범위는 출처가 뒷받침하는 일반 지침이므로, 그때 이 값들을 출발점으로 삼는다.

6. 5. 정리

네 기법은 각각 검색의 다른 단계를 개선한다. 임베딩 학습은 벡터의 품질 자체를 결정하는 뿌리이고, 하이브리드 검색은 키워드와 의미의 사각지대를 서로 메우며, 리랭킹은 추려낸 결과의 순위를 정밀하게 바로잡고, HNSW 튜닝은 검색의 속도와 정확도 사이에서 균형점을 찾는다. 공통된 교훈은 검색이 한 번의 코사인 계산으로 끝나는 것이 아니라 여러 단계로 쌓아 올리는 파이프라인이라는 점이며, 각 단계마다 정확도와 비용의 트레이드오프가 있다는 것이다.

7. 참고 자료

1. [Contrastive learning with hard negatives for sentence embeddings](#)
2. [Text and Code Embeddings by Contrastive Pre-Training \(OpenAI\)](#)
3. [Hybrid Search for RAG: Combining BM25 and Dense Vector Search](#)
4. [Hybrid Search Explained \(Weaviate\)](#)
5. [Bi-Encoders vs Cross-Encoders \(ZeroEntropy\)](#)
6. [Reranker \(BGE documentation\)](#)
7. [Rerankers and Two-Stage Retrieval \(Pinecone\)](#)
8. [Reranking and Cross-Encoders for RAG: BGE, Cohere, Jina](#)
9. [HNSW Indexes with Postgres and pgvector \(Crunchy Data\)](#)
10. [How to optimize performance when using pgvector \(Microsoft Learn\)](#)